INVERTIS
UNIVERSITY BAREILLY

HTML AND CSS LAB ASSIGNMENT (BCS 105)

BTECH 1 SEMESTER

SUBMITTED TO:

Mr Ayush Kumar Yadav

SUBMITTED BY:

Abhijeet Dwivedi
SECTION H
BCS 2026586

SN.	PROGRAM NAME	PAGE	DATE	SIGNATURE
1	Starting a Git Repository	3-5		
2	Pushing and Cloning a Repository	6-9		
3	Your First HTML Page	10-14		
4	Divs, Attributes and a Small Fix	15-21		
5	Creating HTML Pages Using Links and Images	22-27		
6	Types of HTML Lists	28-33		
7	Creating an About Me Page Using Semantic HTML Tags	34-36		
8	Creating Sign-Up Forms and Choices Using HTML	37-42		
9	Creating HTML Tables with Headers and Footers	43-48		
10	CSS Fundamentals: Inline, Internal and External CSS	49-53		
11	Units and CSS Variables	54-60		
12	Study of CSS Box Model, Cards and Scrolling	61-66		
13	Creating Webpage Layouts Using CSS Flexbox	67-73		
14	Creating Webpage Layouts Using CSS Grid	74-78		
15	Creating Responsive Webpages Using CSS Media Queries	79-83		

ASSIGNMENT 1

EXPERIMENT TITLE

Starting a Git Repository

AIM

To start a Git repository for the lab folder, configure user details, create an HTML file, and save changes using Git commits.

SOFTWARE/TOOLS REQUIRED

Visual Studio Code, Git, HTML, Windows Terminal

THEORY

Git is a version control system used to track changes in files. It allows users to create repositories, stage files, commit changes, and view commit history.

ALGORITHM/PROCEDURE

1. Create a folder named html-lab.
2. Initialize Git using git init
3. Set username and email.
4. Create index.html.
5. Add and commit the file.
6. Add a second heading and commit again.
7. View commit history.

PROGRAM/QUERIES/COMMANDS

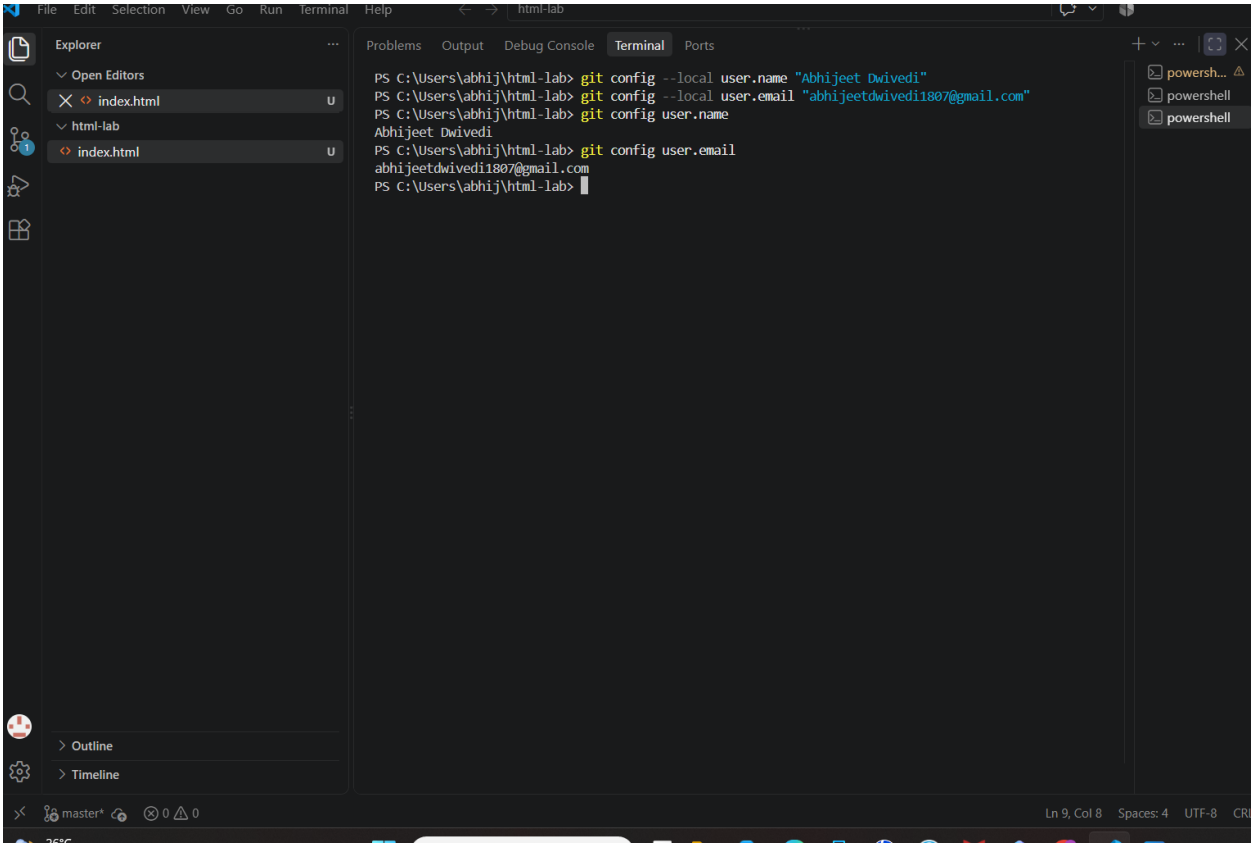
Git commands and HTML code used in the experiment.

INPUT

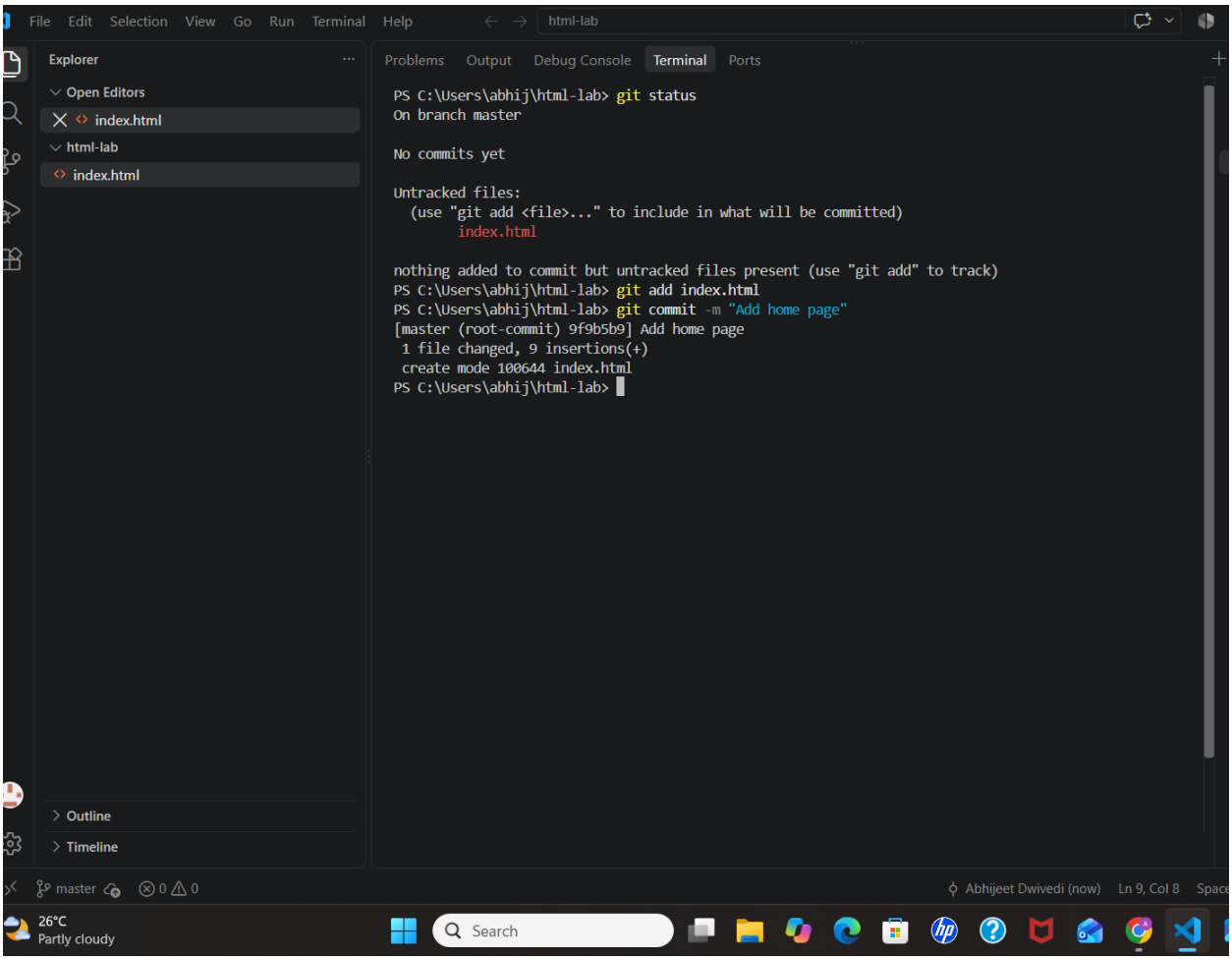
Folder name: html-lab. File name: index.html. Headings: Welcome to My Home Page and Next Meeting.

OUTPUT

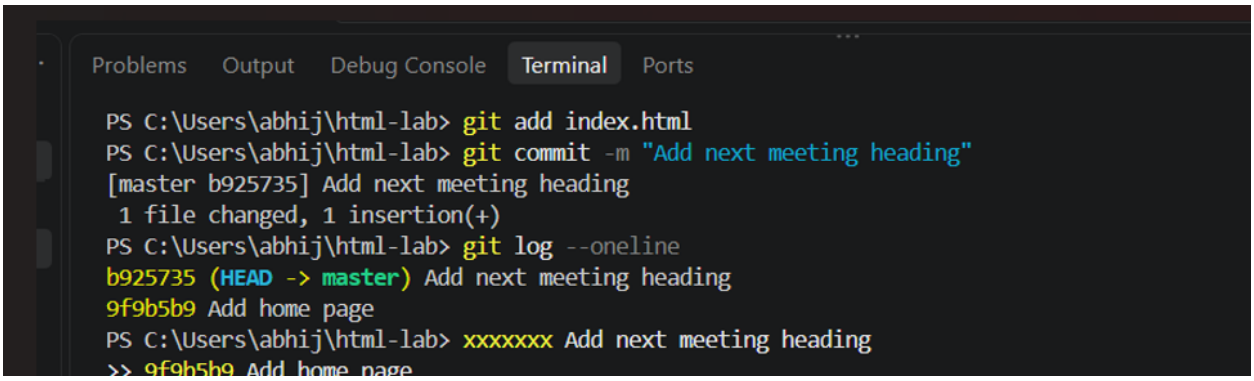
Screenshot 1 — task configuration



SCREENSHOT 2 — FIRST COMIT



SCREENSHOT 3 — GIT LOG



RESULT

The experiment was performed successfully. A Git repository was created, user details were configured, an HTML file was added, and changes were saved using Git commits. The commit history was also verified successfully.

ASSIGNMENT 2

EXPERIMENT TITLE

Pushing and Cloning a Repository

1.AIM

Start a Git repository for your lab folder and save your work with commits.

2.SOFTWARE/TOOLS REQUIRED

- Visual Studio Code
- Git
- Windows Terminal
- HTML

3. THEORY

Git is a distributed version control system used to track changes in files. Git remote is used to connect a local repository to another repository. Git push sends commits to the remote repository, while Git clone creates a copy of a repository with its commit history.

4.ALGORITHM/PROCEDURE

- Open the existing `html-lab` folder.
- Create a bare repository named `html-remote`.
- Add the remote repository as `origin`.
- Verify the remote using `git remote -v`.
- Push the commits to the remote repository.
- Clone the remote repository into a new folder named `html-copy`.
- View the commit history using `git log –oneline`

5. PROGRAM/QUERIES/COMMANDS

TASK 1: CREATE A REMOTE REPOSITORY AND ADD ORIGIN

```
git init --bare ../html-remote
git remote add origin ../html-remote
git remote –v
```

TASK 2: PUSH COMMIT TO ORIGIN

```
Git push origin master
```

TASK 3 : CLONE THE REPOSITORY AND VIEW COMMIT HISTORY

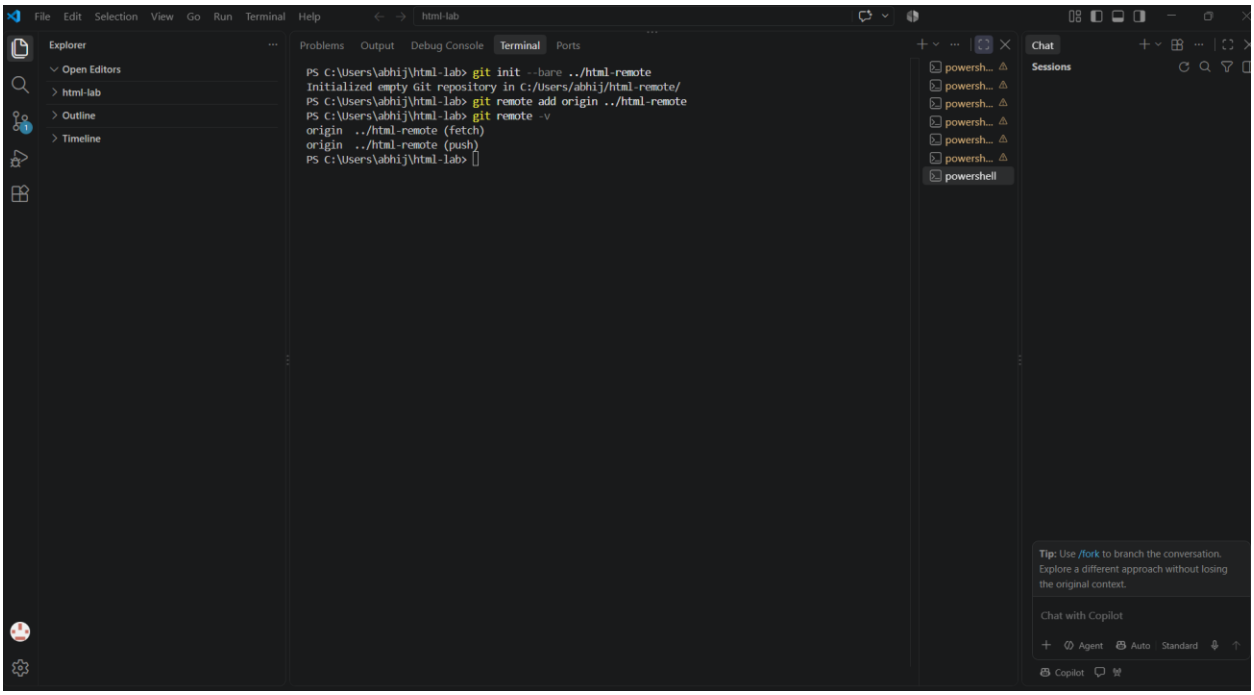
```
cd ..
git clone html-remote html-copy
cd html-copy
git log –oneline
```

7.INPUT

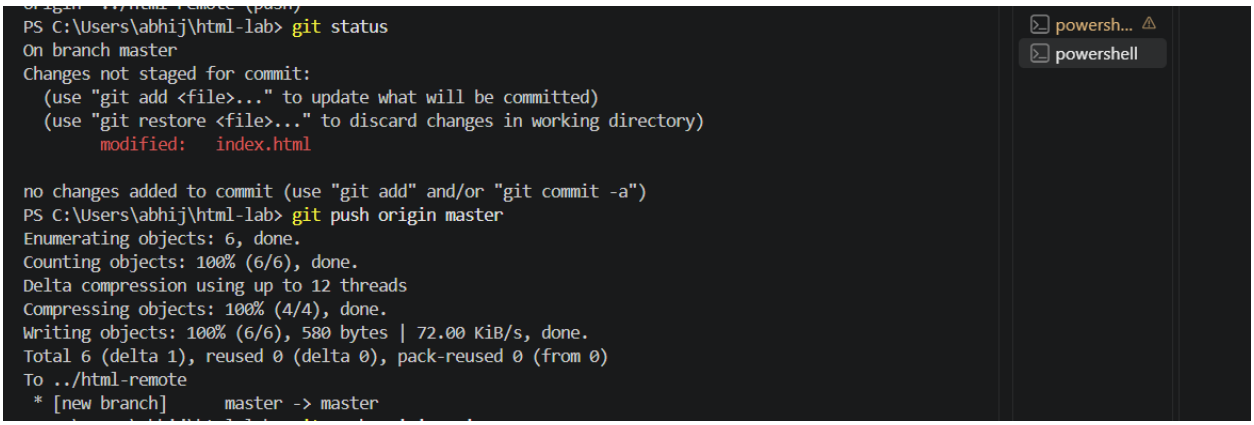
- Repository name: html-remote
- Clone folder name: html-copy
- Existing commits:
- Add home page
- Add next meeting headin

8.OUTPUT

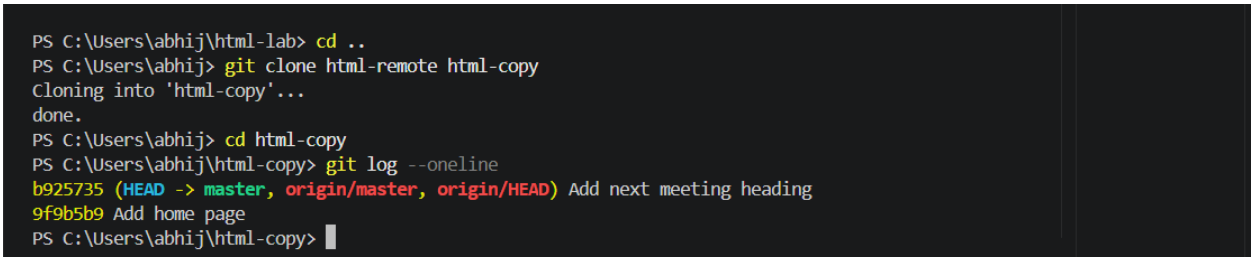
TASK 1 SCREENSHOT



TASK 2 SCREENSHOT



TASK 3 SCREENSHOT



9.RESULT

The experiment was performed successfully. A bare Git repository was created and added as a remote repository. The commits were pushed to the remote repository, and a clone was created successfully. The commit history was verified in the cloned repository.

ASSIGNMENT 3

EXPERIMENT TITLE

Your First HTML Page

1.AIM

Write a first HTML page with a title, headings and a paragraph, and format some text.

2.SOFTWARE/TOOLS REQUIRED

- Visual Studio Code
- HTML
- Live Server extension
- Web browser

3. THEORY

HTML (HyperText Markup Language) is used to create the structure of web pages. It contains different tags such as headings, paragraphs, and text formatting tags.

The `<h1>`, `<h2>`, and `<h3>` tags are used to create headings of different sizes. The `<p>` tag is used to create paragraphs. The `<strong>` tag makes text bold, while the `<em>` tag makes text italic. The `<br>` tag is used to insert a line break.

4. ALGORITHM/PROCEDURE

- Open Visual Studio Code.
- Create a folder named `html-lab`.
- Create the file `first.html`.
- Write HTML code to display a title and heading.
- Create `headings.html` and add three headings and a paragraph.
- Create `text-format.html` and add two paragraphs with text formatting.
- Save all files.
- Open each HTML file in the browser using Live Server.
- Observe the output and take screenshots.

5.PROGRAM/QUERIES/COMMANDS

TASK 1: first.html

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Page</title>
</head>
<body>
  <h1>Hello, I am Asha.</h1>
</body>
</html>
```

TASK 2: headings.html

```
<!DOCTYPE html>
<html>
<head>
  <title>HTML Headings</title>
</head>
<body>
  <h1>This is Heading 1</h1>

  <h2>This is Heading 2</h2>

  <h3>This is Heading 3</h3>
```

```
  <p>This is a paragraph written below the headings.</p>
</body>
</html>
```

TASK 3:text-format.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Text Formatting</title>
</head>
<body>
  <p>
    HTML is a <strong>powerful</strong> language used
    to create web pages.
    <br>
    It helps us design the structure of a website.
  </p>

  <p>
    Learning HTML is <em>interesting</em> and useful
    for web development.
  </p>
</body>
</html>
```

7. INPUT

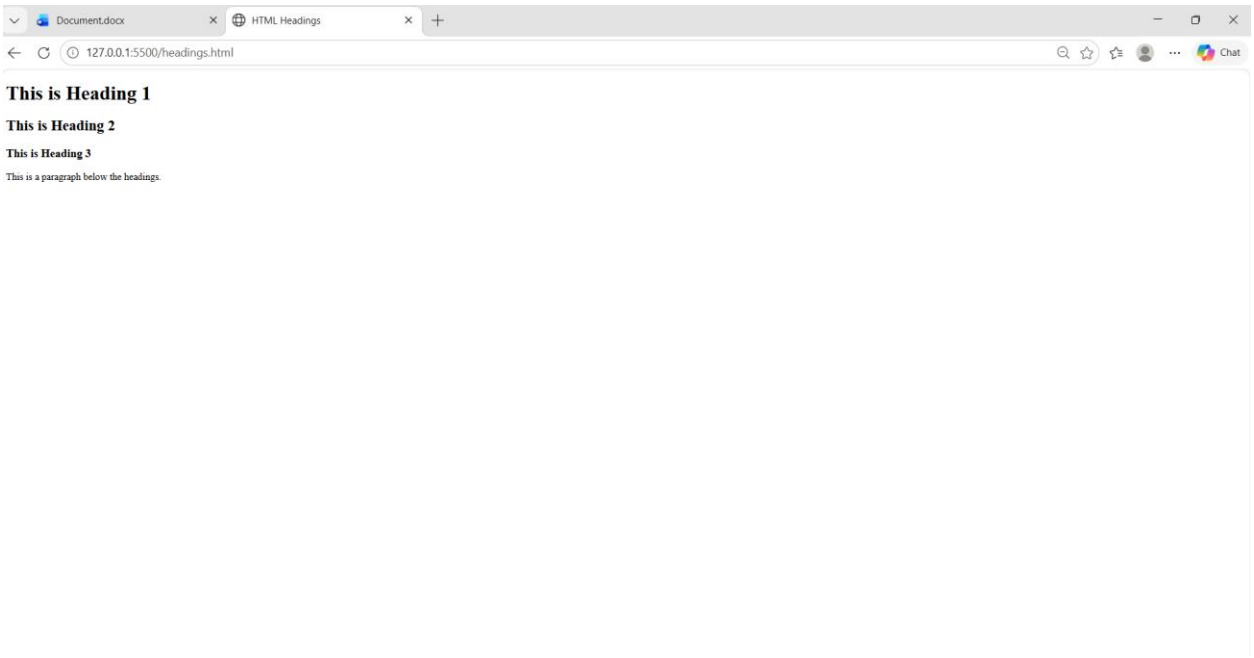
No external input is required. The HTML code is written directly in the files.

8.OUTPUT

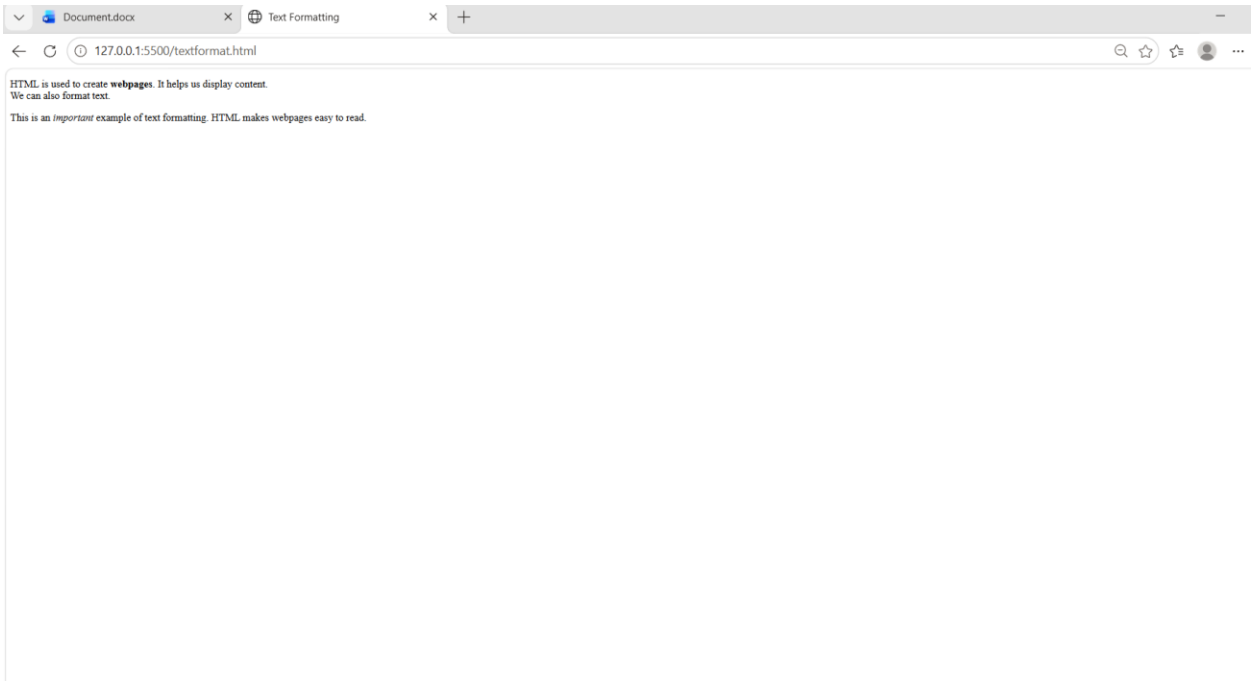
TASK 1 SCREENSHOT



TASK 2 SCREENSHOT



TASK 3 SCREENSHOT



RESULT

The experiment was performed successfully. HTML pages were created using headings, paragraphs, bold text, italic text and line breaks.

ASSIGNMENT 4

EXPERIMENT TITLE

Divs, Attributes and a Small Fix

1.AIM

Group parts of a page with div elements, add a data attribute, and fix the errors in a broken page.

2.SOFTWARE/TOOLS REQUIRED

- Visual Studio Code
- HTML
- Live Server extension
- Web browser

3.THEORY

The `<div>` tag is used to group HTML elements into sections or blocks. It helps organize the content of a web page.

HTML attributes provide additional information about HTML elements. The `class` attribute is used to assign a class name to an element, while the `id` attribute gives an element a unique identity. The `data-*` attribute is used to store custom data in an HTML element.

HTML tags must be written correctly, with proper opening and closing tags, to ensure that the web page displays as expected.

4.ALGORITHM/PROCEDURE

- Open Visual Studio Code.
- Open the existing `html-lab` folder.
- Create `layout.html`.
- Write three div blocks with the classes header, main and footer.
- Create `attributes.html`.
- Add a div with the id profile and a paragraph with the data-topic attribute.
- Create `attendance-fixed.html`.
- Correct the errors in the provided broken HTML code.
- Save all files.
- Open each HTML file in a browser using Live Server.

- Observe the output and take screenshots.

5.PROGRAM/QUERIES/INPUT

TASK 1 : layout.html

```
<!DOCTYPE html>

<html>

<head>

  <title>Div Layout</title>

</head>

<body>


  <div class="header">

    This is the Header

  </div>


  <div class="main">

    This is the Main Content

  </div>


  <div class="footer">

    This is the Footer

  </div>
```

```
</body>

</html>
```

TASK 2: attributes.html

```
<!DOCTYPE html>

<html>

<head>

  <title>HTML Attributes</title>

</head>

<body>


  <div id="profile">

    <p data-topic="hobbies">

      My hobbies are reading, playing games and listening to music.

    </p>

  </div>


</body>

</html>
```

TASK 3: attendance-fixed.html

```
<!DOCTYPE html>

<html>

<head>
```

```
  <title>Attendance</title>
</head>
<body>

  <h1>Attendance Report</h1>

  <p>Asha: 92%</p>

  <p>Rohan: 85%</p>

</body>
</html>
```

ERROR FIXED

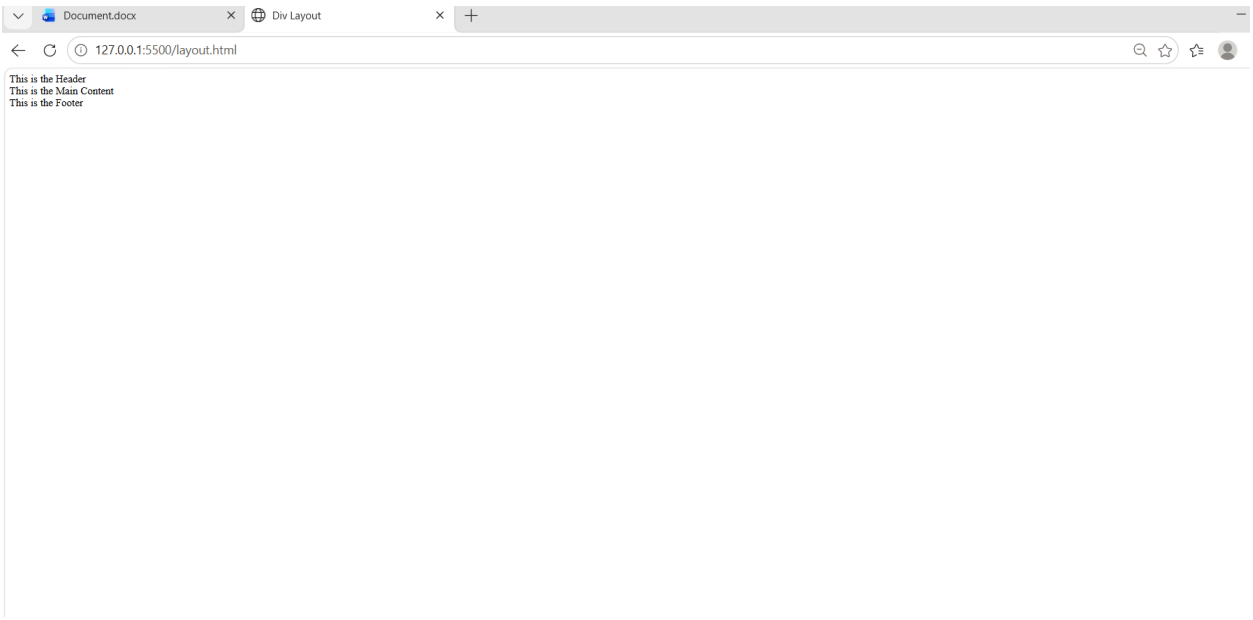
Incorrect code	Correct code
`<title>Attendance</title>` was missing the opening <	`<title>Attendance</title>`
`<h1>Attendance Report` was missing closing tag	`<h1>Attendance Report</h1>`
`<p>Asha: 92%>` was missing closing tag	`<p>Asha: 92%</p>`
`<p>Rohan: 85%/p>` had incorrect closing tag	`<p>Rohan: 85%</p>`

7.INPUT

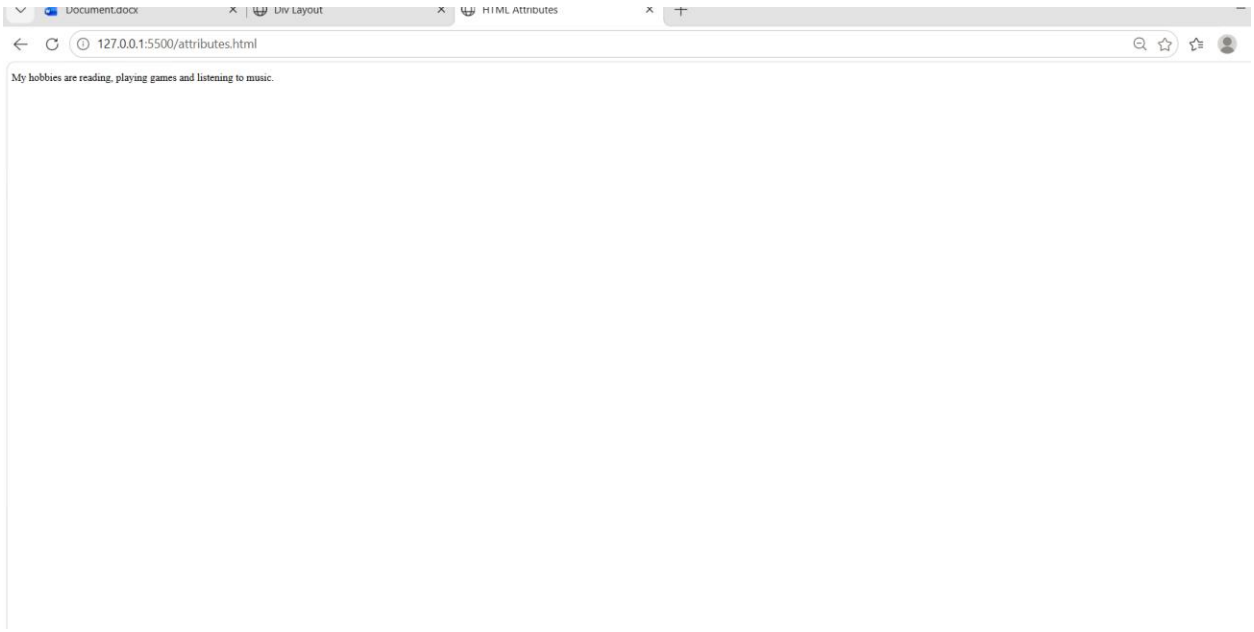
HTML code containing div elements, attributes and a broken HTML page.

8.OUTPUT

TASK 1 SCREENSHOT



TASK 2 SCREENSHOT



TASK 3 SCREENSHOT



9.RESULT

The experiment was performed successfully. HTML pages were created using div elements and attributes. The errors in the broken HTML page were identified and corrected, and the output was displayed successfully in the web browser.

ASSIGNMENT 5

EXPERIMENT TITLE

Creating HTML Pages Using Links and Images.

1.AIM

To create HTML pages, link two pages together using hyperlinks, and insert an image with appropriate alternative text.

2. SOFTWARE/TOOLS REQUIRED

- Visual Studio Code
- HTML
- Web Browser (Google Chrome)
- Live Server Extension

3.THEORY

HTML (HyperText Markup Language) is used to create web pages. The anchor tag `<a>` is used to create hyperlinks between different web pages. The image tag `<img>` is used to display images on a webpage.

The `href` attribute specifies the destination of a link, while the `src` attribute specifies the location of an image. The `alt` attribute provides alternative text if the image cannot be displayed.

4.ALGORITHM/PROCEDURE

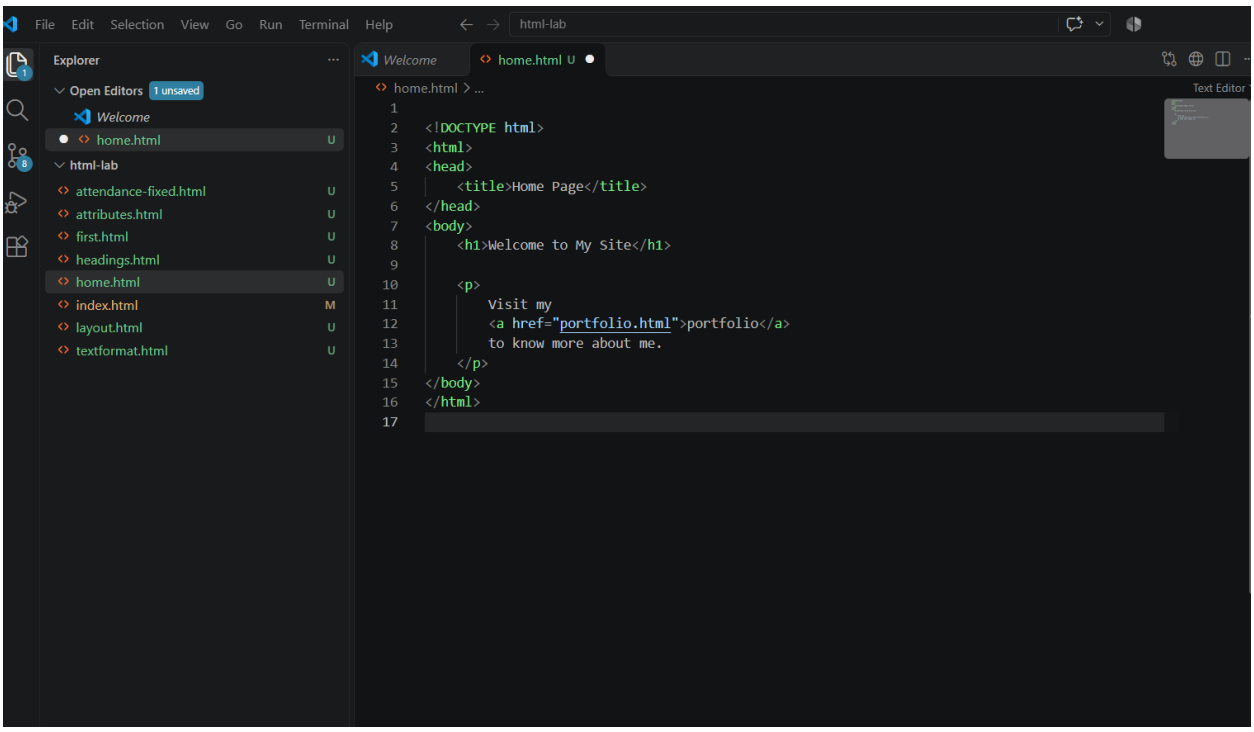
- Open Visual Studio Code.
- Create a folder named `html-lab`.
- Create a file named `home.html`.
- Add a heading, paragraph, and link to `portfolio.html`.
- Create a file named `portfolio.html`.
- Add a heading and link back to `home.html`.
- Create a folder named `images` inside `html-lab`.
- Add the image provided by the instructor to the images folder.
- Create a file named `portfolio-image.html`.
- Add a heading and insert the image with width and height of 200 pixels.

- Add suitable alternative text to the image.
- Save all files.
- Open each HTML file in the browser using Live Server.
- Observe the output and take screenshots.

5.PROGRAM/QUERIES/COMMANDS

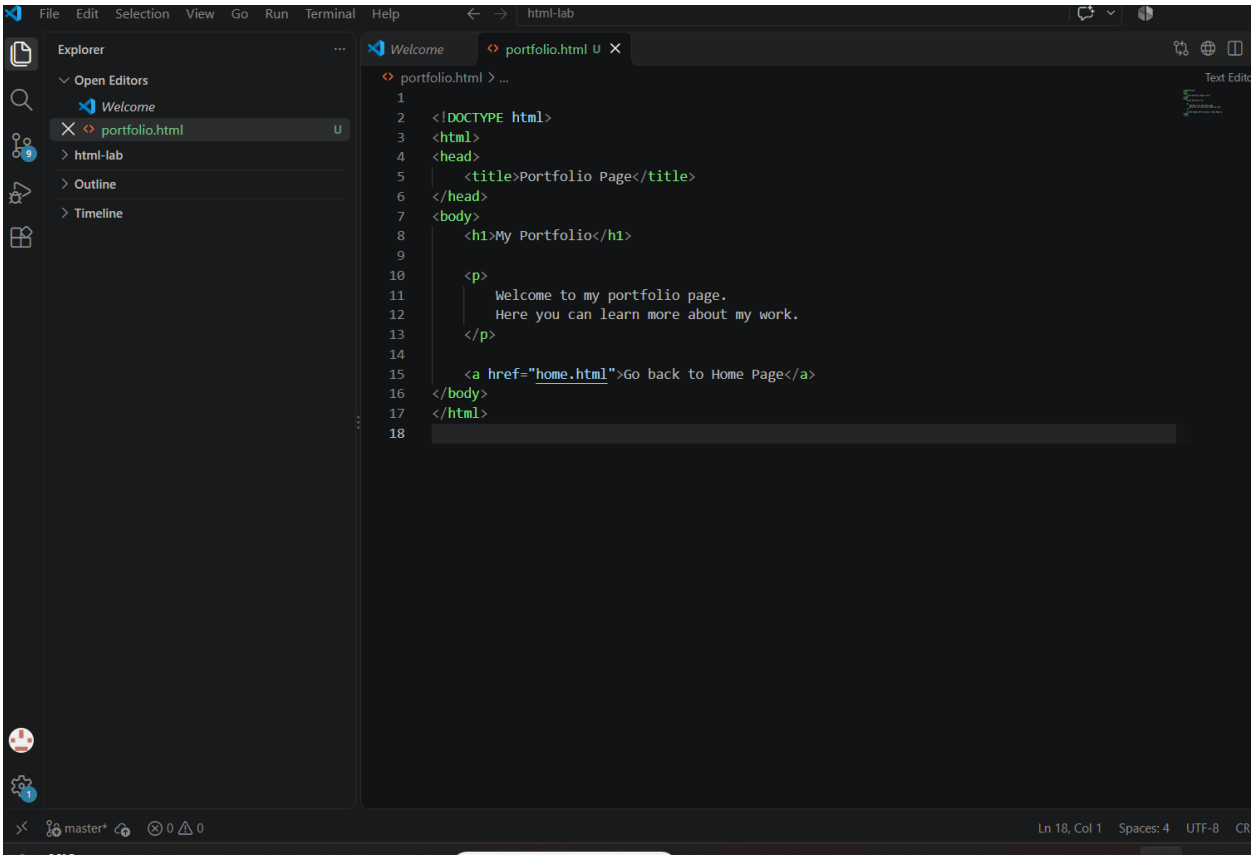
Task 1: Create home.html

Create a file named `home.html` and write the following code:



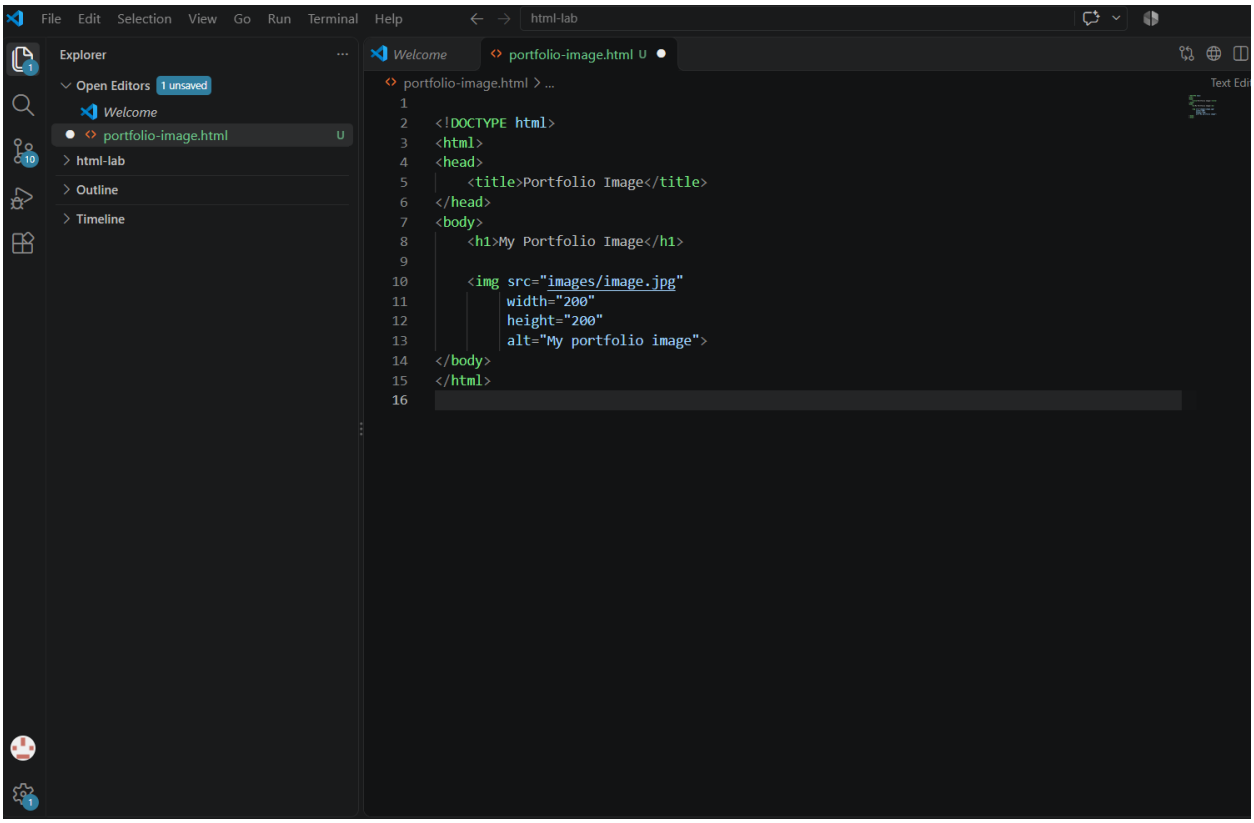
Task 2: Create portfolio.html

Create a file named `portfolio.html` and write the following code



Task 3: Create portfolio-image.html

Create a file named `portfolio-image.html` and write the following code:



7.INPUT

- HTML code for creating three web pages.
- Hyperlinks to connect home and portfolio pages.
- An image file provided by the instructor.

8.OUTPUT

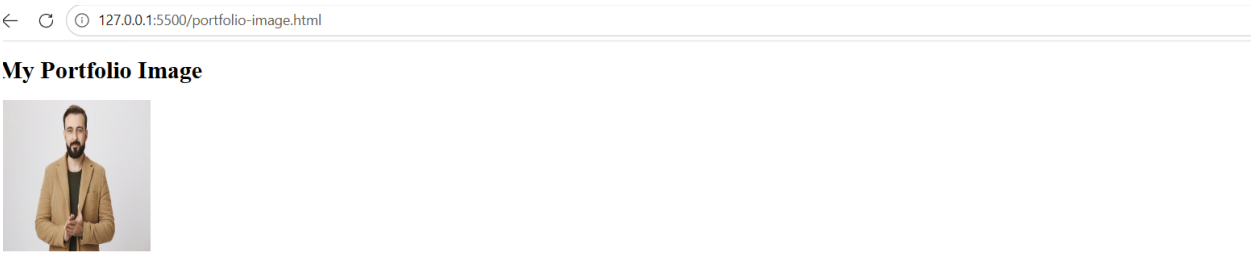
TASK 1 SCREENSHOT



TASK 2 SCREENSHOT



TASK 3 SCREENSHOT



8.RESULT

The experiment was performed successfully. Three HTML pages were created, hyperlinks were used to navigate between pages, and an image was inserted with appropriate width, height, and alternative text.

ASSIGNMENT 6

EXPERIMENT TITLE

Types of HTML Lists

1.AIM

To create a bulleted list, a numbered list with a nested list, and a description list using HTML.

2.SOFTWARE/TOOLS REQUIRED

- Visual Studio Code
- HTML
- Web Browser (Google Chrome)
- Live Server Extension

3.THEORY

HTML provides different types of lists to organize and display information on a webpage.

- Unordered List (`<ul>`): Displays items with bullet points.
- Ordered List (`<ol>`): Displays items with numbers.
- Nested List: A list placed inside another list.
- Description List (`<dl>`): Displays terms and their descriptions using `<dt>` and `<dd>` elements.

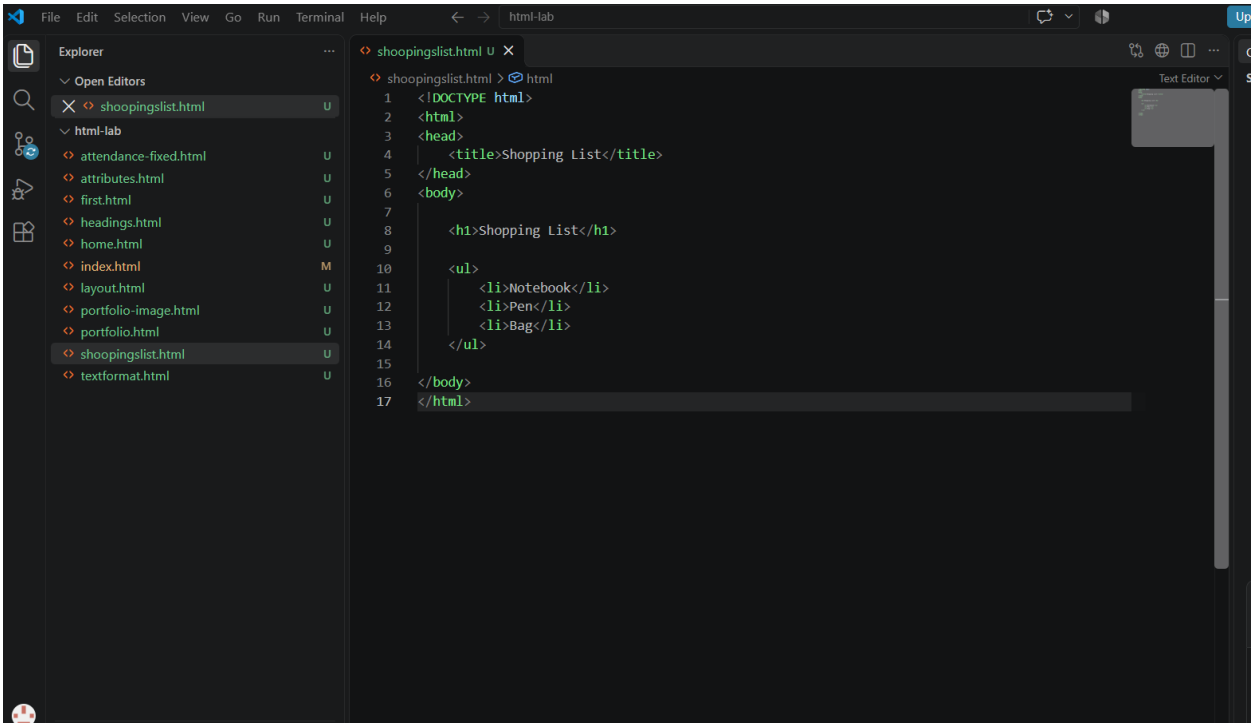
4.ALGORITHM/PROCEDURE

- Open Visual Studio Code.
- Create a folder named `html-lab`.
- Create a file named `shopping-list.html`.
- Write HTML code to create a heading and an unordered list of three items.
- Create a file named `canteen-menu.html`.
- Write HTML code to create a numbered list with a nested numbered list.
- Create a file named `faq.html`.
- Write HTML code to create a description list containing two questions and their answers.

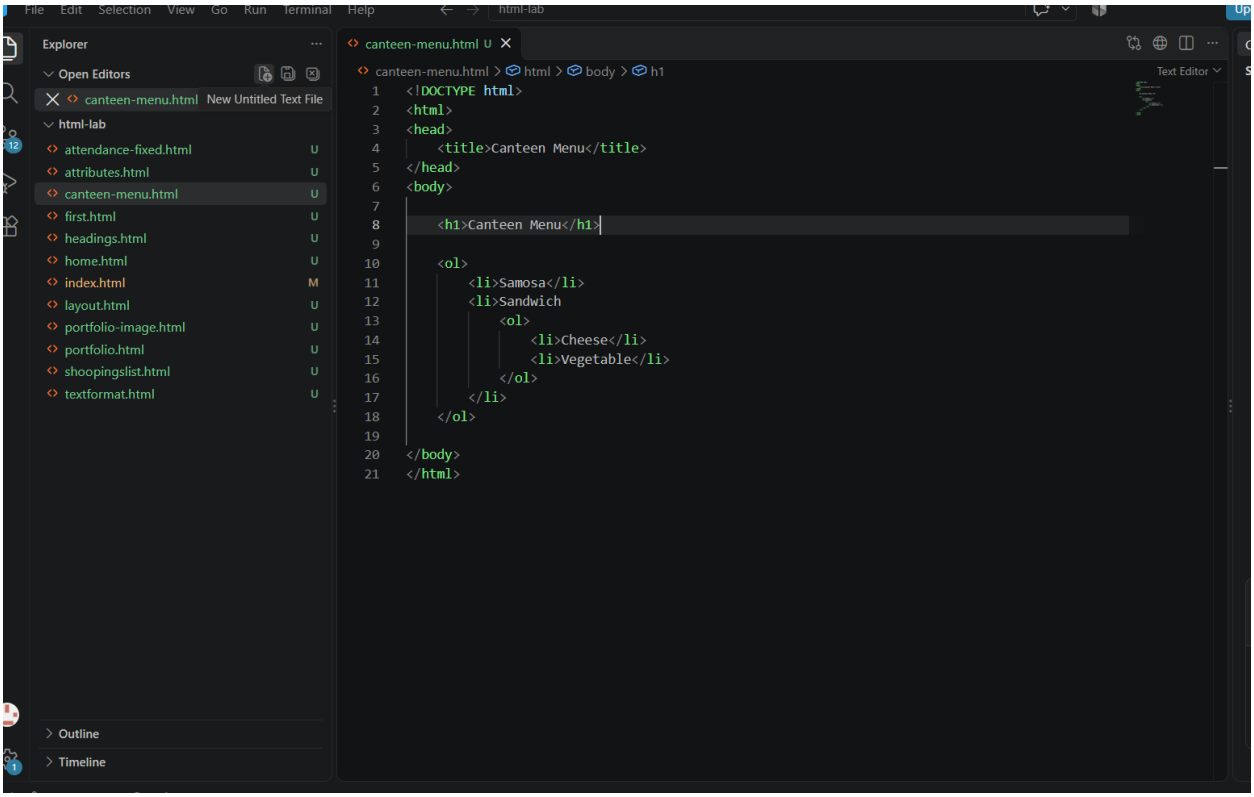
- Save all files.
- Open each HTML file in the browser using Live Server.
- Observe the output and take screenshots.

5.PROGRAM/QUERIES/COMMANDS

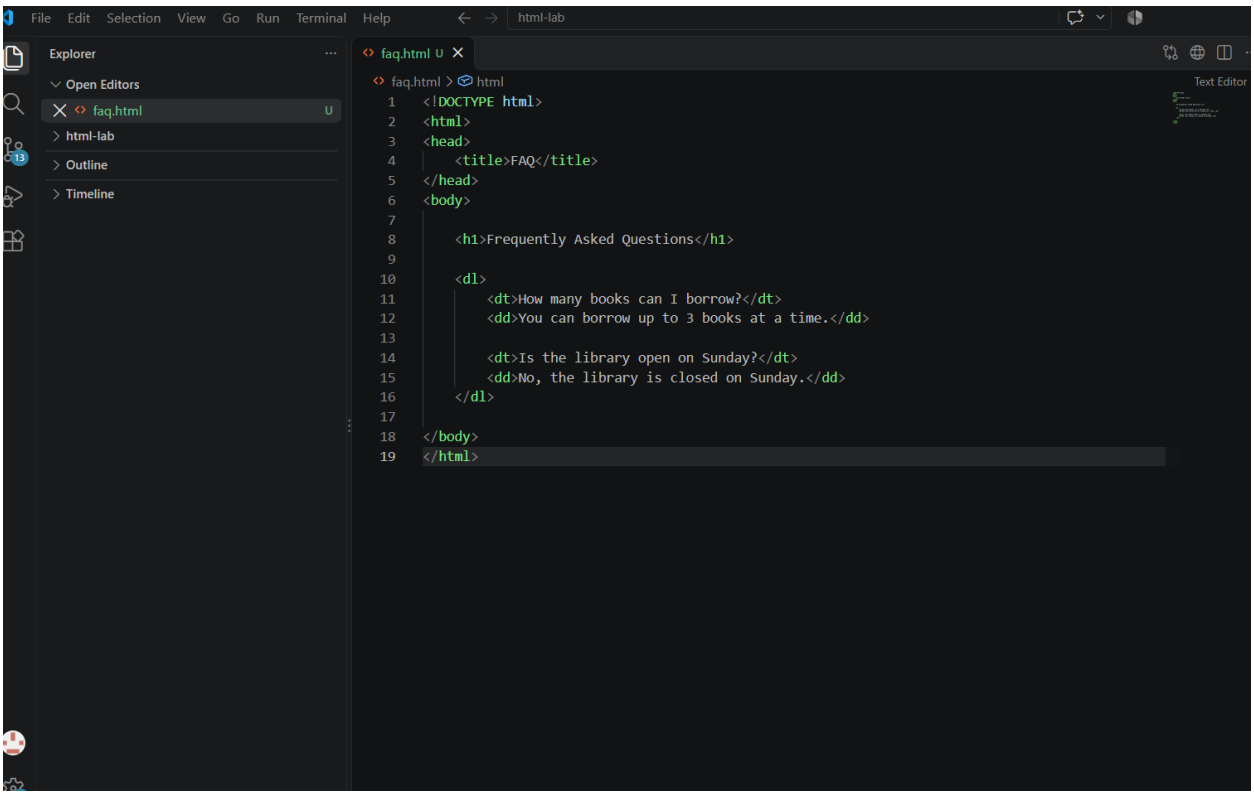
TASK 1 : shoppings-list.html



TASK 2 : canteen-menu.html



TASK 3 : faq.html



6.INPUT

Task 1: Shopping list items: Notebook, Pen, and Bag.

Task 2: Canteen menu items: Samosa and Sandwich. Nested items under Sandwich: Cheese and Vegetable.

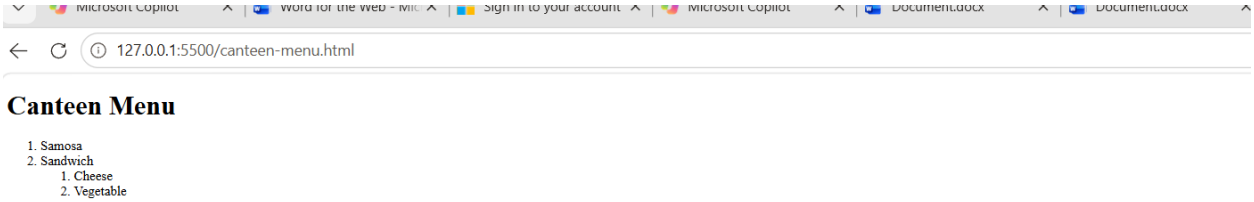
Task 3: Questions: How many books can I borrow? Is the library open on Sunday?

7.OUTPUT

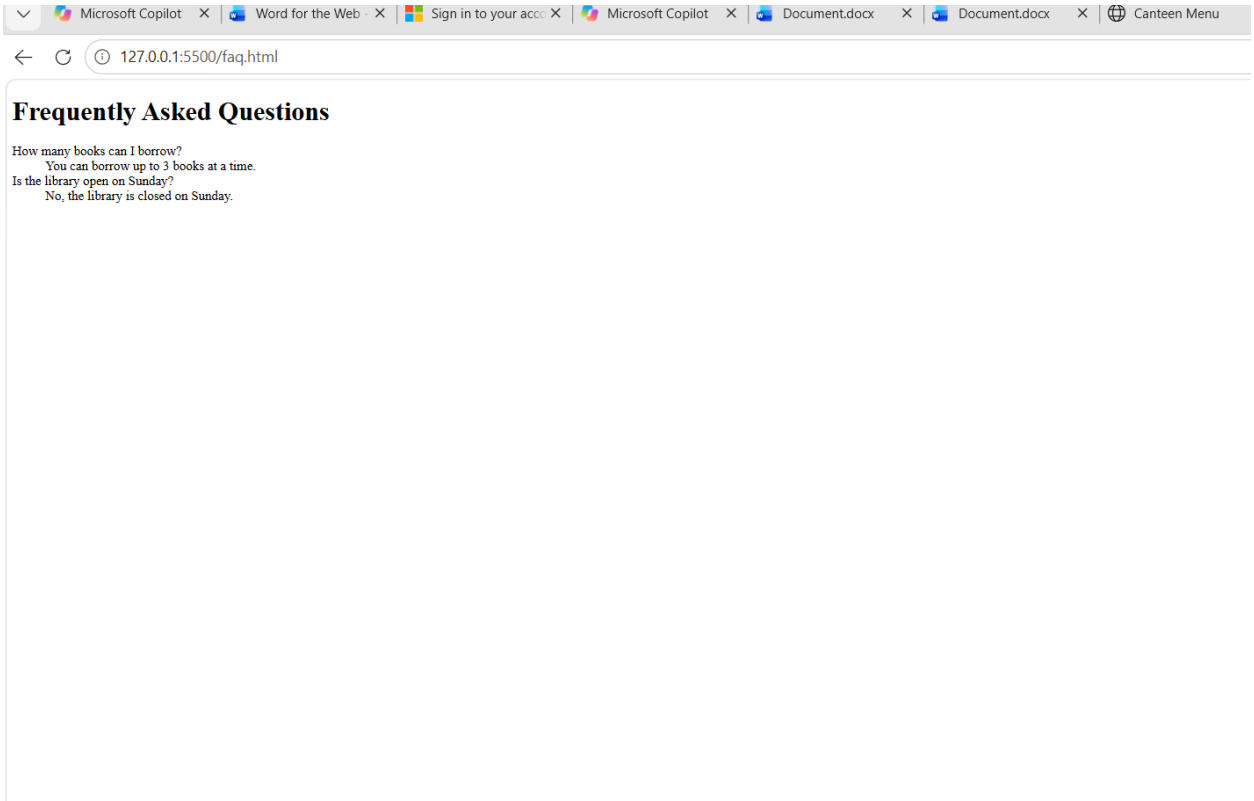
TASK 1: A webpage displaying a shopping list with three bullet points.



TASK 2: A webpage displaying a numbered canteen menu with a nested numbered list.



TASK 3: A webpage displaying two questions with their respective answers.



8.RESULT

The experiment was performed successfully. Different types of HTML lists, including unordered lists, ordered lists, nested lists, and description lists, were created and displayed in the web browser.

ASSIGNMENT 7

EXPERIMENT TITLE

Creating an About Me Page Using Semantic HTML Tags

1.AIM

To create a short About Me webpage using semantic HTML tags, an About section, navigation links, and a contact email link.

2.SOFTWARE/TOOLS REQUIRED

- Visual Studio Code
- HTML
- Web Browser (Google Chrome)
- Live Server Extension

3.THEORY

HTML semantic tags provide meaning and structure to a webpage.

- `<header>`: Defines the header section of a webpage.
- `<nav>`: Defines navigation links.
- `<main>`: Contains the main content of the webpage.
- `<section>`: Defines a particular section of content.
- `<footer>`: Defines the footer section.
- `<a>`: Creates a hyperlink to another section or webpage.
- `mailto:`: Allows users to click an email address to send an email.

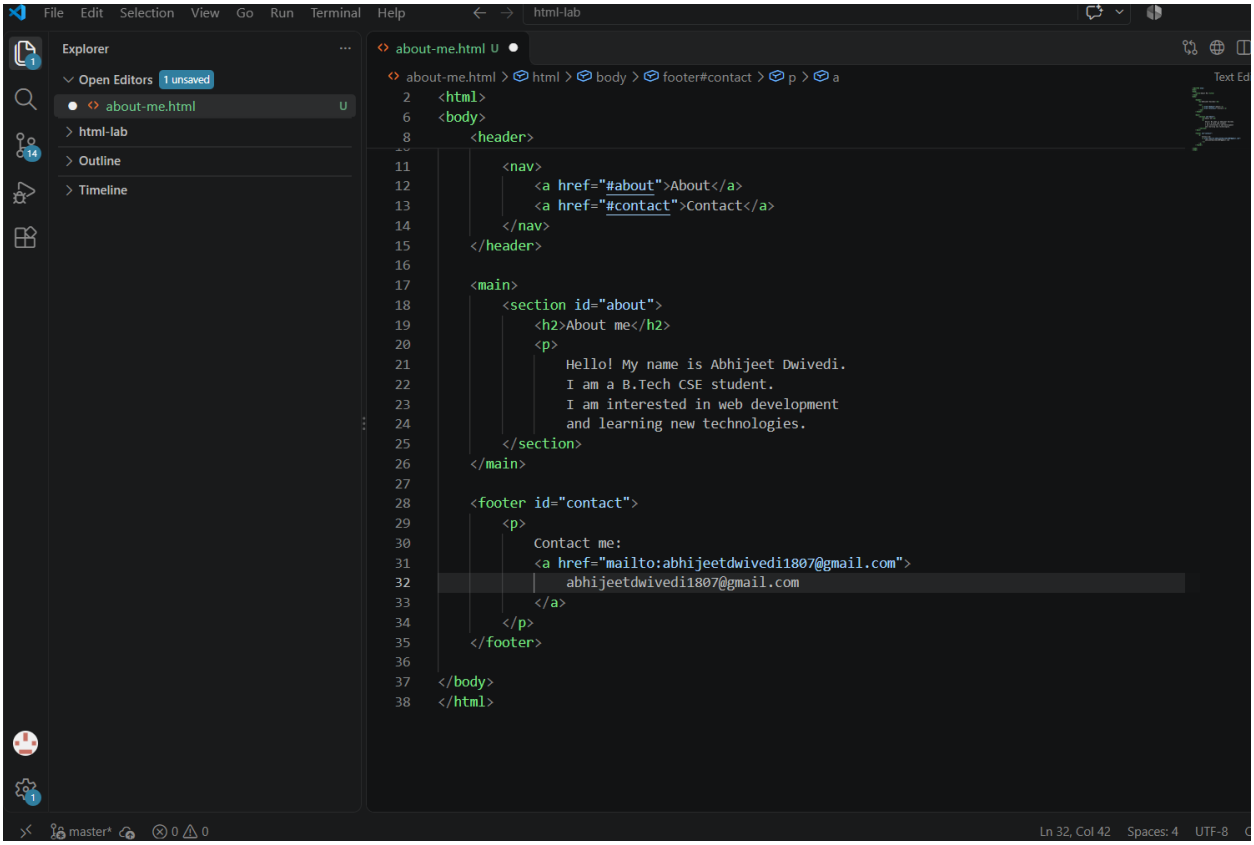
4.ALGORITHM/PROCEDURE

- Open Visual Studio Code.
- Open the `html-lab` folder.
- Create a new file named `about-me.html`.
- Write the basic HTML document structure.
- Create a header containing your name and two navigation links.
- Add a main section with an About Me heading and paragraph.
- Create a footer with a clickable email address.

- Save the file.
- Open the HTML file in the browser using Live Server.
- Observe the output and take a screenshot.

5.PROGRAM/QUERIES/COMMANDS

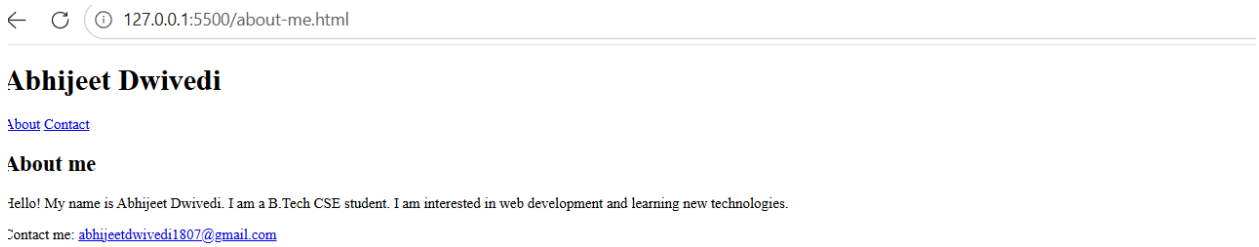
About-me.html



6.INPUT

- Name: Abhijeet Dwivedi
- About Me information: B.Tech CSE student interested in web development and learning new technologies.
- Email address: abhijeetdwivedi133@gmail.com

7.OUTPUT



8.RESULT

The experiment was performed successfully. An About Me webpage was created using semantic HTML tags, navigation links, an About section, and a clickable email address.

ASSIGNMENT 8

EXPERIMENT TITLE

Creating Sign-Up Forms and Choices Using HTML

1.AIM

To create labelled forms with text, email, and password fields, radio buttons, dropdown menus, and a submit button using HTML.

2.SOFTWARE TOOLS REQUIRED

- Visual Studio Code
- HTML
- Web Browser (Google Chrome)
- Live Server Extension

3.THEORY

HTML forms are used to collect information from users.

- `<form>`: Creates a form to collect user input.
- `<label>`: Provides a name for an input field.
- `<input type="text">`: Creates a text input field.
- `<input type="email">`: Creates an email input field.
- `<input type="password">`: Creates a password input field.
- `<input type="radio">`: Creates radio buttons where only one option can be selected.
- `<select>`: Creates a dropdown menu.
- `<option>`: Defines options inside a dropdown.
- `required`: Makes a field compulsory.
- `<button>`: Creates a button to submit the form.

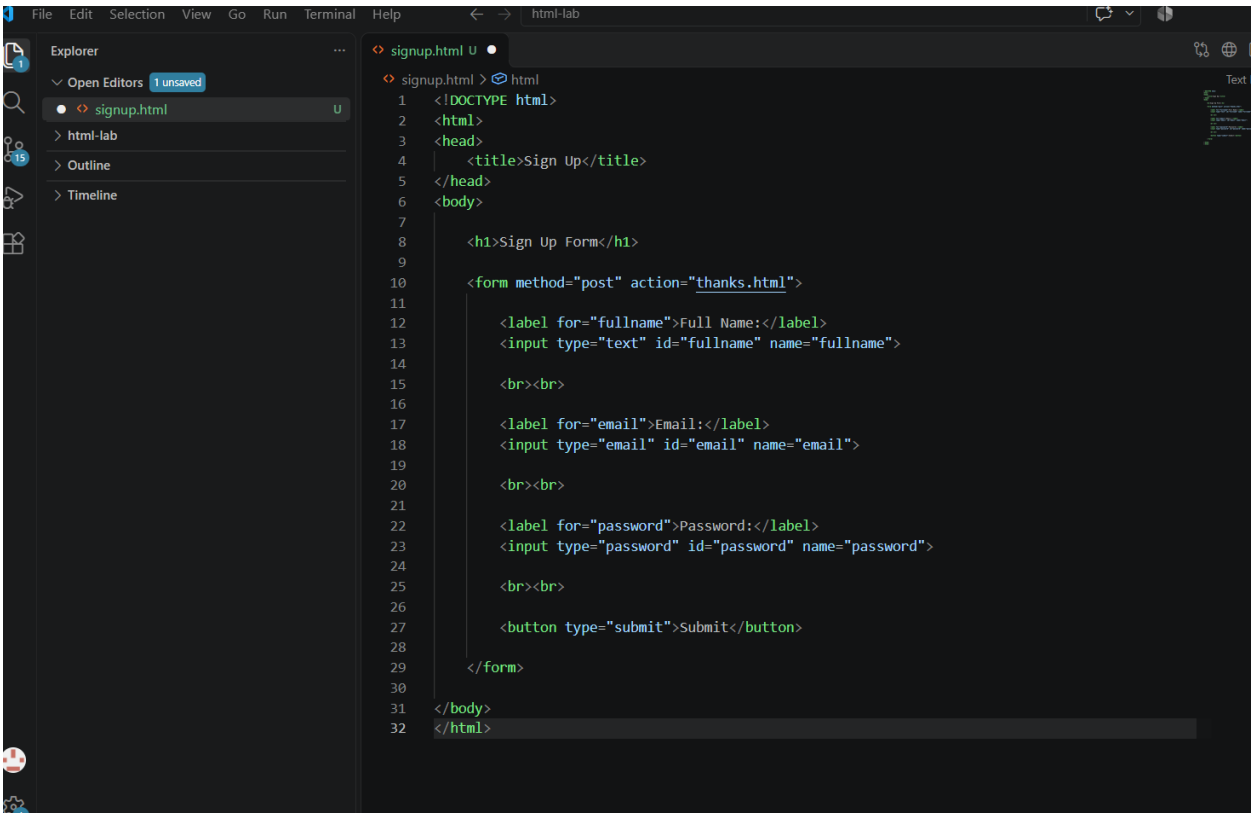
4. ALGORITHM/PROCEDURE

- Open Visual Studio Code.
- Open the `html-lab` folder.
- Create a file named `signup.html`.

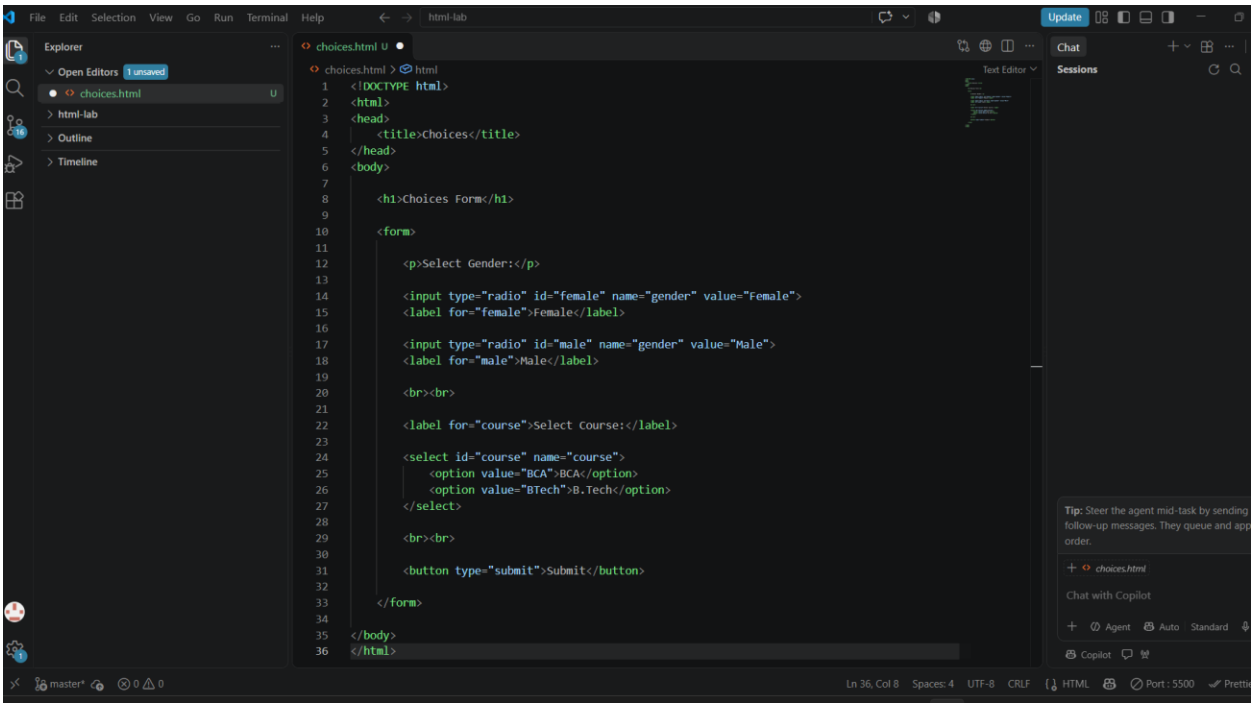
- Write HTML code to create a form with full name, email, and password fields.
- Create a file named `choices.html`.
- Write HTML code to create gender radio buttons and a course dropdown.
- Create a file named `signup-required.html`.
- Write HTML code to create required name and email fields with a submit button.
- Save all files.
- Open each HTML file in the browser using Live Server.
- Observe the output and take screenshots.

5.PROGRAM/QUERIES/INPUT

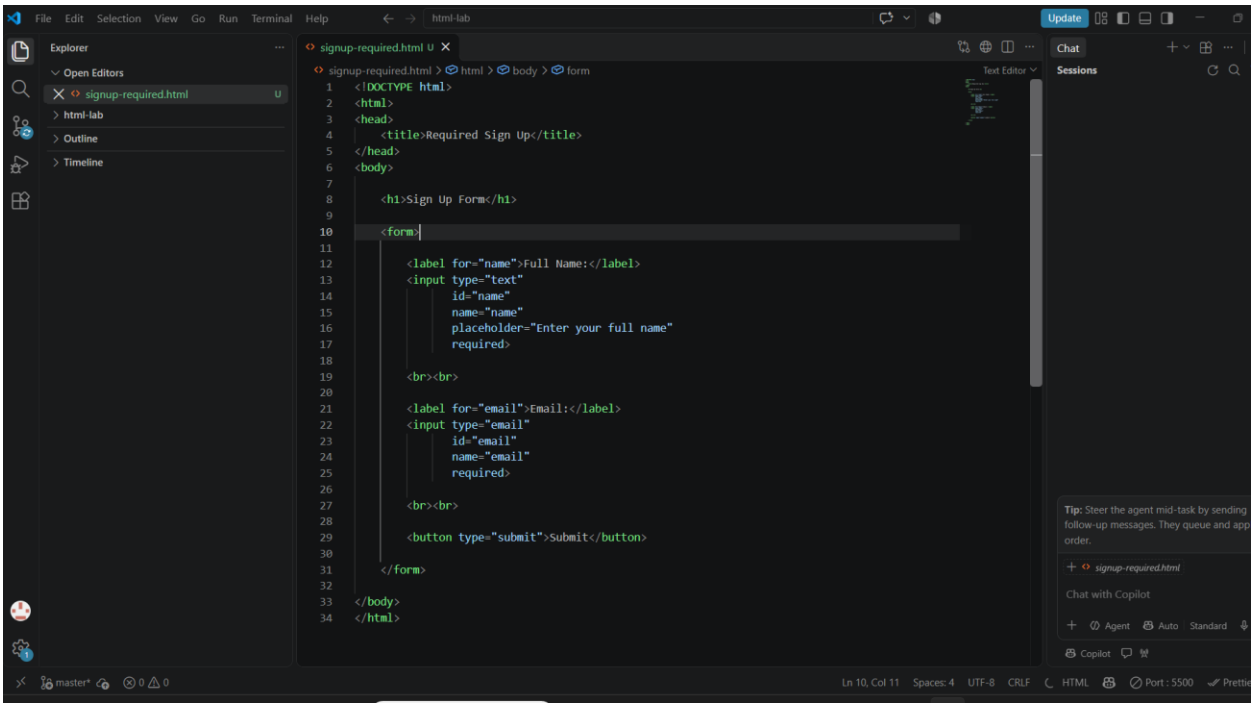
TASK 1 : signup.html



TASK 2 : choices.html



TASK 3: signup-required.html



6.INPUT

Task 1:

- Full Name: Abhijeet Dwivedi
- Email: Your email address
- Password: Your password

Task 2:

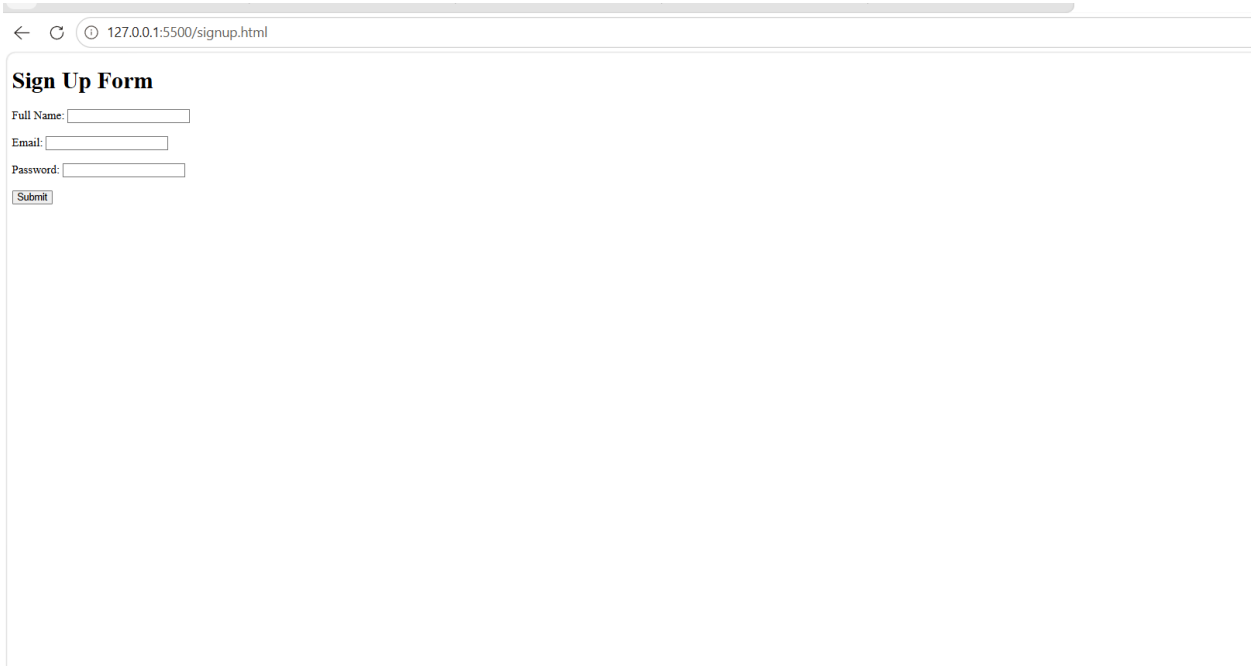
- Gender: Female or Male
- Course: BCA or B.Tech

Task 3:

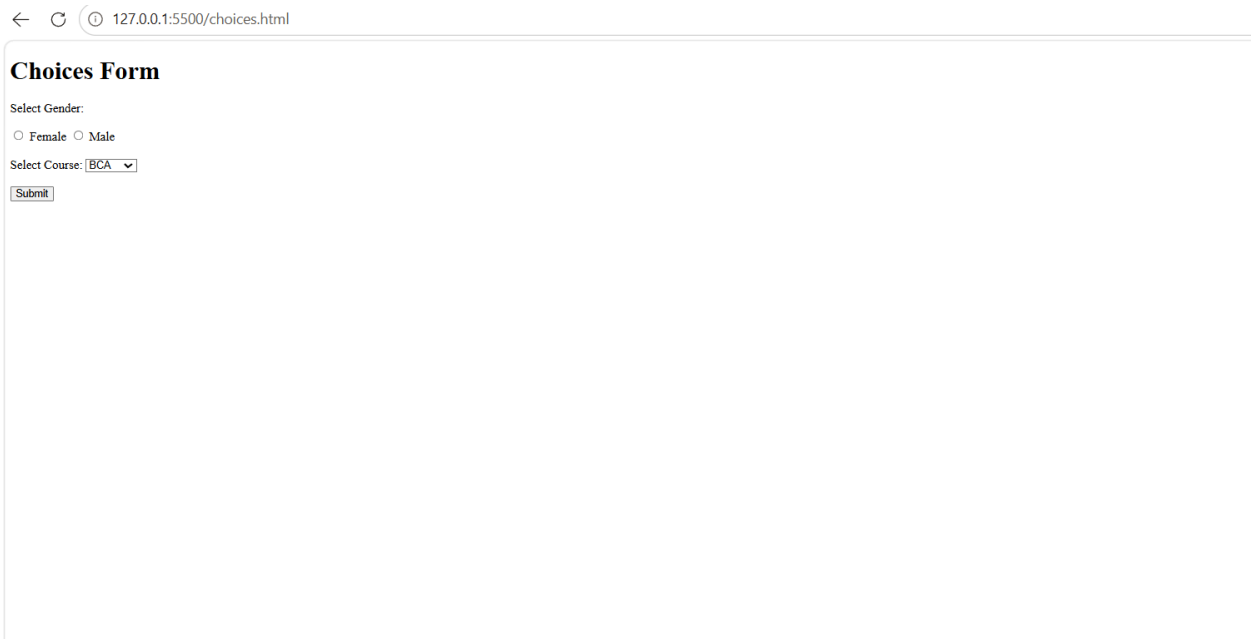
- Full Name: Abhijeet Dwivedi
- Email: Your email address

7.OUTPUT

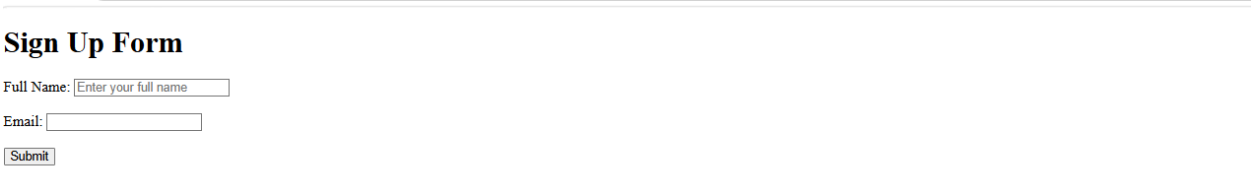
TASK 1 : A sign-up form displaying full name, email, and password fields.



TASK 2 : A form displaying two gender radio buttons and a course dropdown.



TASK 3: A form displaying required name and email fields with a submit button.



8.RESULT

The experiment was performed successfully. Different HTML form elements, including text fields, email fields, password fields, radio buttons, dropdown menus, and required fields, were created and displayed in the web browser.

ASSIGNMENT 9

EXPERIMENT TITLE

Creating HTML Tables with Headers and Footers

1.AIM

To create HTML tables with captions, headers, rows, footers, and scope attributes.

2. SOFTWARE/TOOLS REQUIRED

- Visual Studio Code
- HTML
- Web Browser (Google Chrome)
- Live Server Extension

3.THEORY

HTML tables are used to display data in rows and columns.

- `<table>`: Creates a table.
- `<caption>`: Adds a title to the table.
- `<thead>`: Defines the header section.
- `<tbody>`: Defines the body of the table.
- `<tfoot>`: Defines the footer section of the table.
- `<tr>`: Defines a table row.
- `<th>`: Defines a table heading.
- `<td>`: Defines a table data cell.
- `scope="col"`: Specifies that a heading applies to a column.
- `scope="row"`: Specifies that a heading applies to a row.

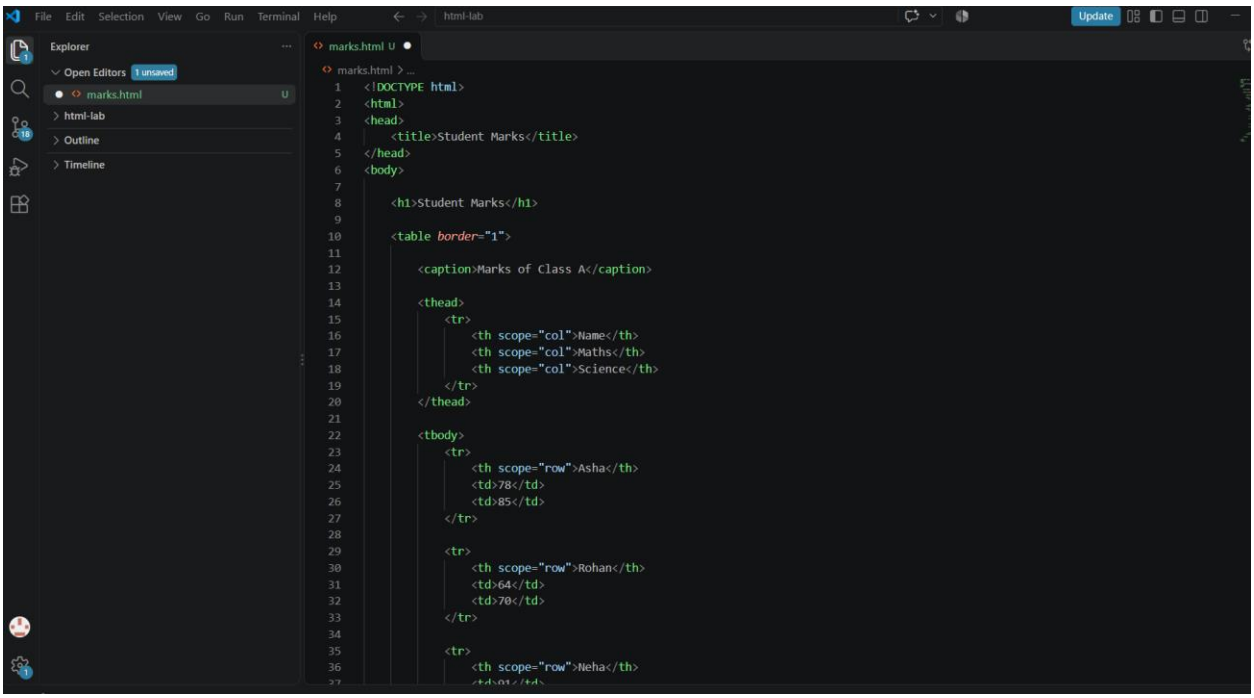
4.ALGORITHM/PROCEDURE

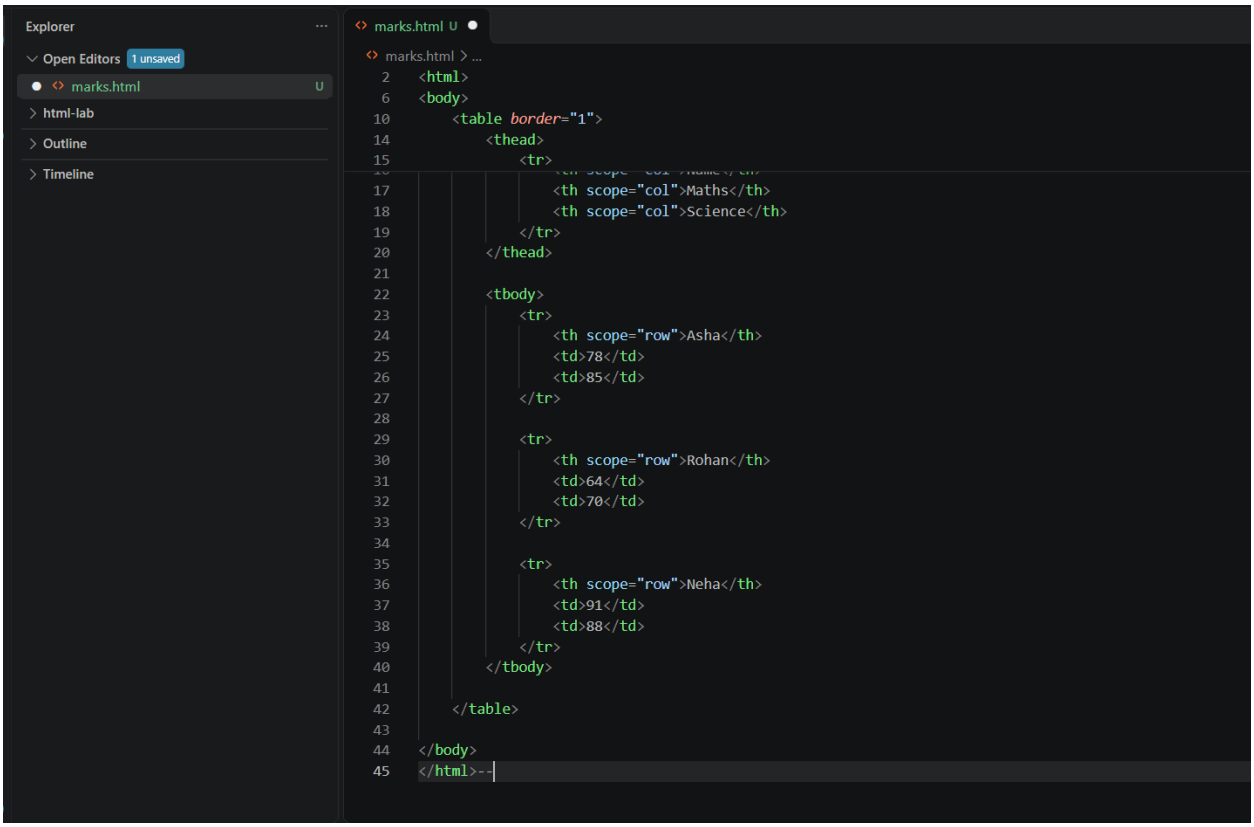
- Open Visual Studio Code.
- Open the `html-lab` folder.
- Create a file named `marks.html`.
- Write HTML code to create a table with a caption, header row, and three student rows.

- Create another file named `marks-average.html`.
- Write HTML code to create a table with two student rows and a footer showing the class average.
- Save both files.
- Open each HTML file in the browser using Live Server.
- Observe the output and take screenshots.

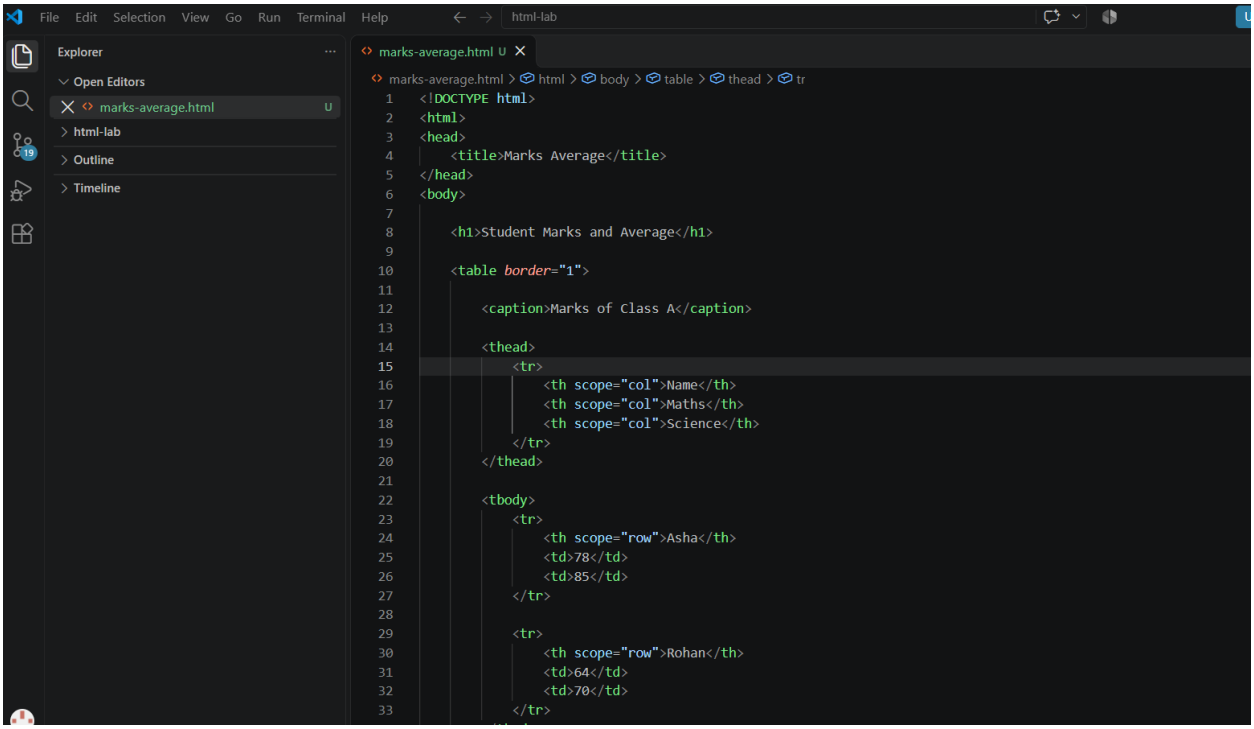
5.PROGRAM/QUERIES/COMMANDS

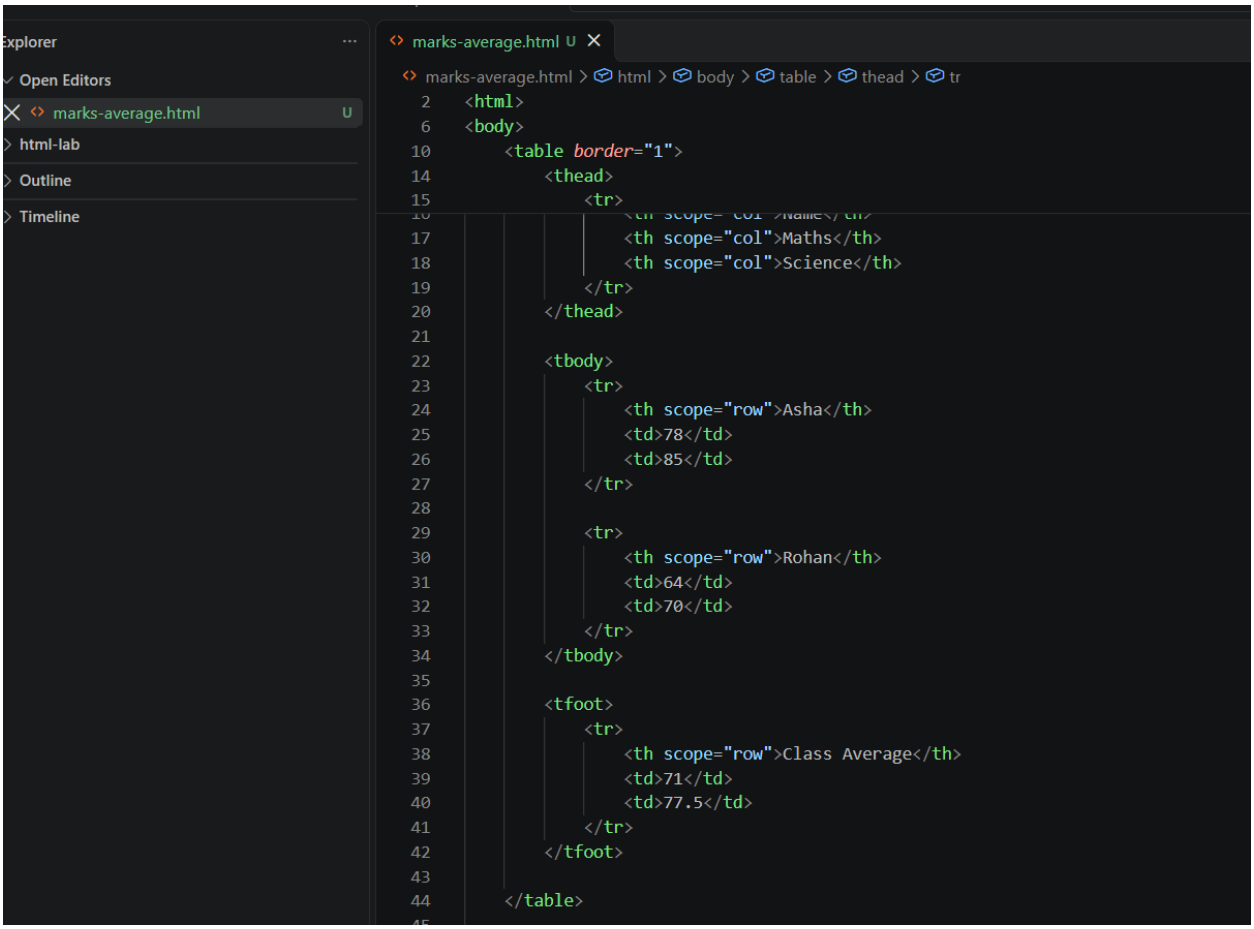
TASK 1 : marks.html





TASK 2:marks-average.html





6.INPUT

Task 1:

Student marks:

- Asha: Maths 78, Science 85
- Rohan: Maths 64, Science 70
- Neha: Maths 91, Science 88

Task 2:

Student marks:

- Asha: Maths 78, Science 85
- Rohan: Maths 64, Science 70

Class averages:

- Maths: 71
- Science: 77.5

7.OUTPUT

TASK 1: A table displaying the marks of three students with a caption and header row.

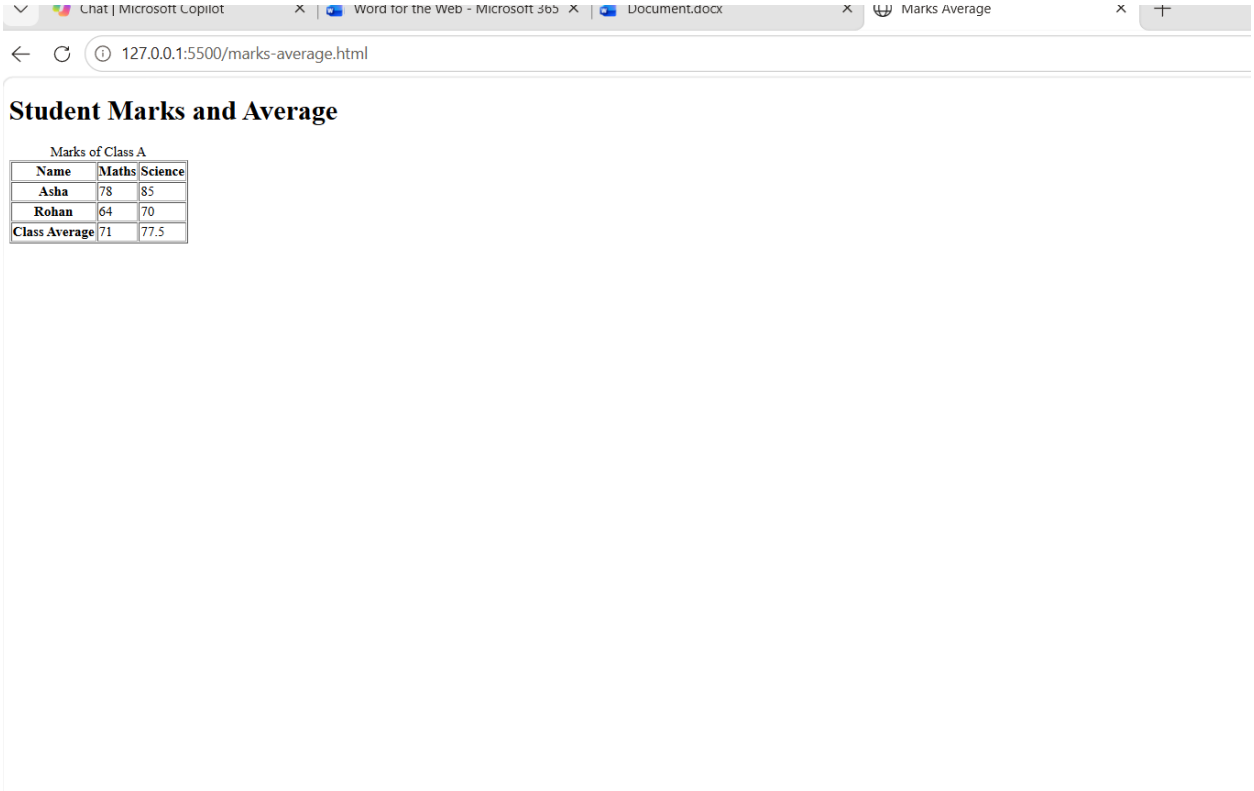


Student Marks

Marks of Class A

Name	Maths	Science
Asha	78	85
Rohan	64	70
Neha	91	88

TASK 2 : A table displaying the marks of two students with a footer row showing the class averages.



8.RESULT

The experiment was performed successfully. HTML tables were created using captions, headers, student rows, and footer rows. Scope attributes were also used to identify row and column headings.

ASSIGNMENT 10

EXPERIMENT TITLE

To study CSS fundamentals and apply inline, internal, and external CSS using selectors and text styles.

1.AIM

To apply CSS in three ways (inline, internal, and external) and use selectors to style HTML elements.

2.SOFTWARE/TOOLS REQUIRED

- Visual Studio Code
- HTML
- CSS
- Live Server
- Web Browser

3.THEORY

CSS (Cascading Style Sheets) is used to style HTML webpages. It controls the appearance of HTML elements, including their color, font size, background, alignment, and other visual properties.

There are three ways to apply CSS:

1. Inline CSS: CSS is written inside the HTML element using the `style` attribute.
2. Internal CSS: CSS is written inside the `<style>` tag in the HTML document's `<head>` section.
3. External CSS: CSS is written in a separate `.css` file and linked to the HTML document.

CSS selectors are used to select HTML elements for styling. Common selectors include element selectors, class selectors, and ID selectors.

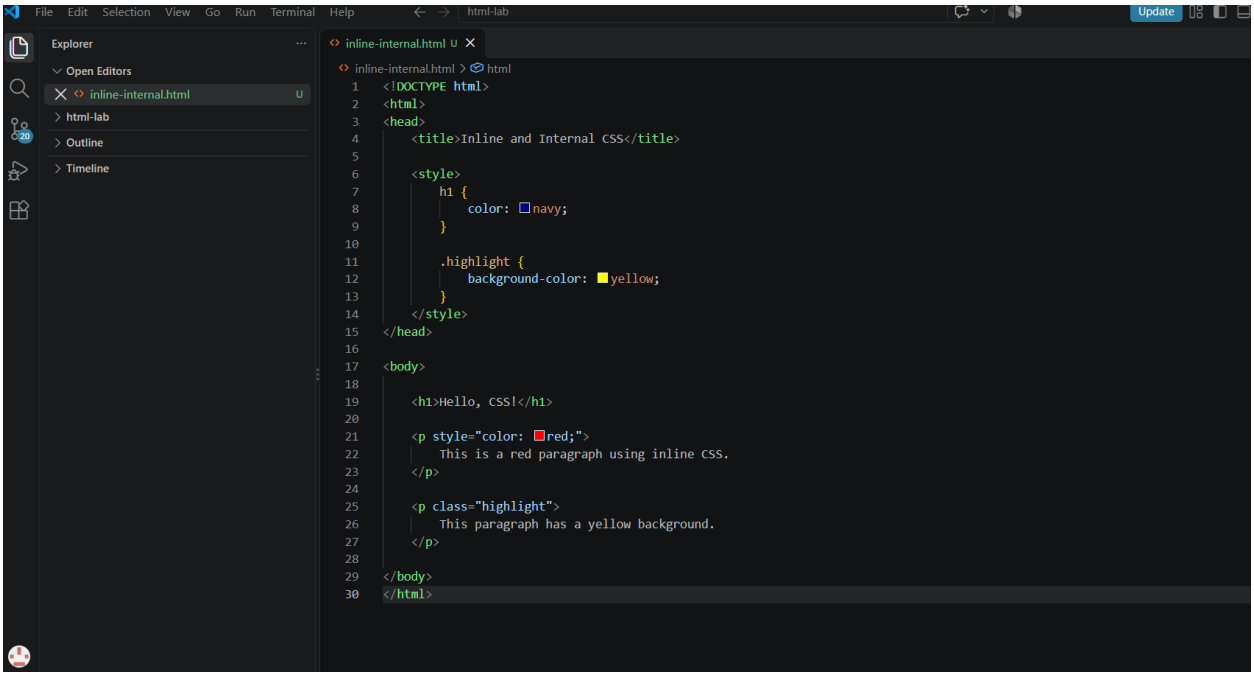
4.ALGORITHM/PROCEDURE

- Open Visual Studio Code.

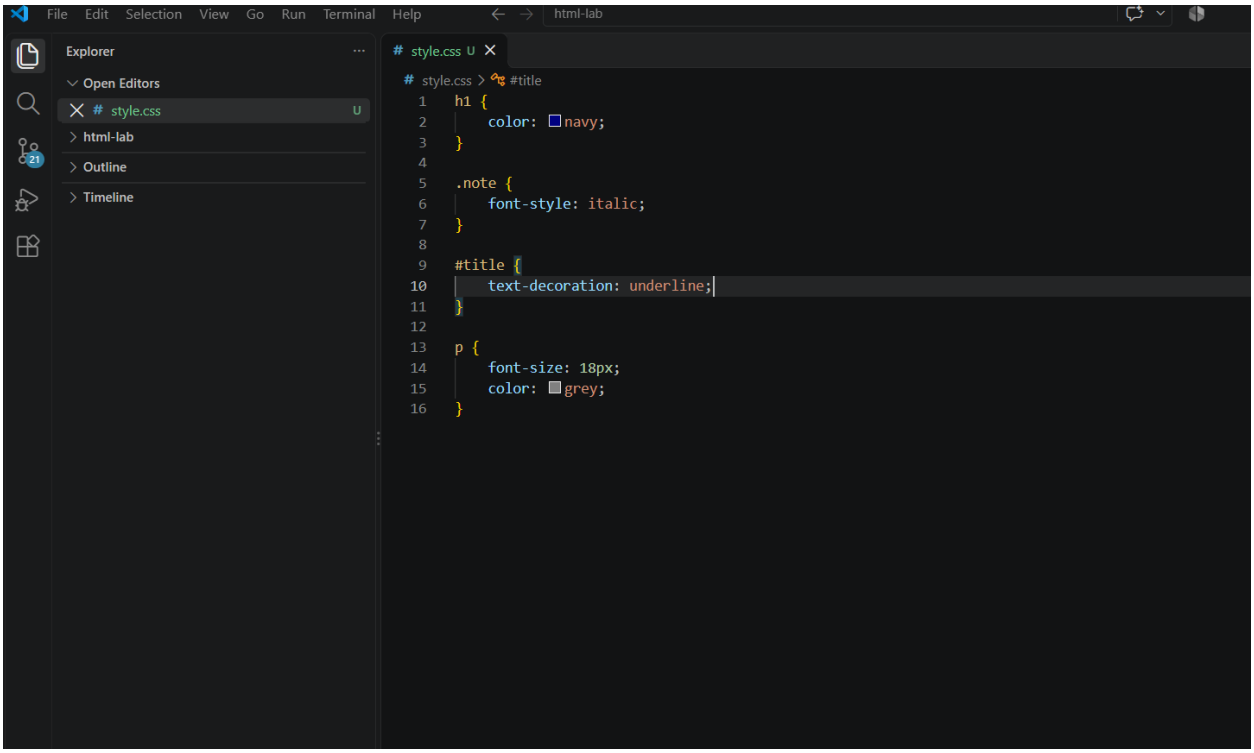
- Open the `html-lab` folder.
- Create a file named `inline-internal.html`.
- Write HTML code using internal and inline CSS.
- Create a file named `style.css`.
- Write CSS rules for headings, paragraphs, classes, and IDs.
- Create a file named `linked.html`.
- Link the external CSS file to `linked.html`.
- Save all files.
- Open each HTML file in the browser using Live Server.
- Observe the output and take screenshots.

5.PROGRAM/QUERIES/COMMANDS

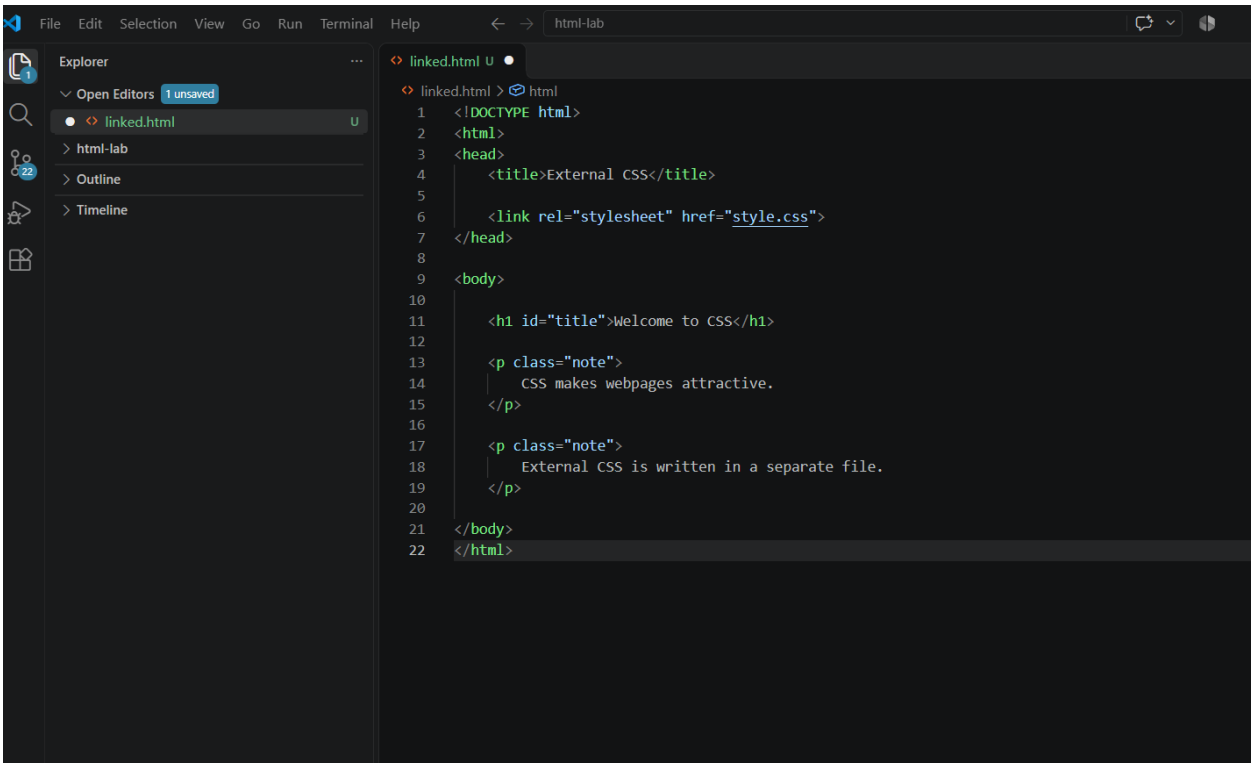
TASK 1 : create inline-internal.html



TASK 2 : create style.css



TASK 3 : create linked.html

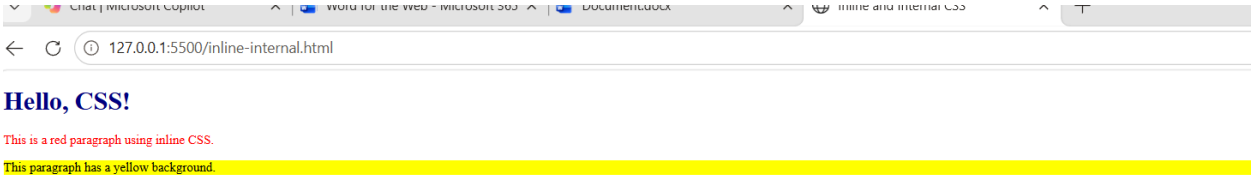


6.INPUT

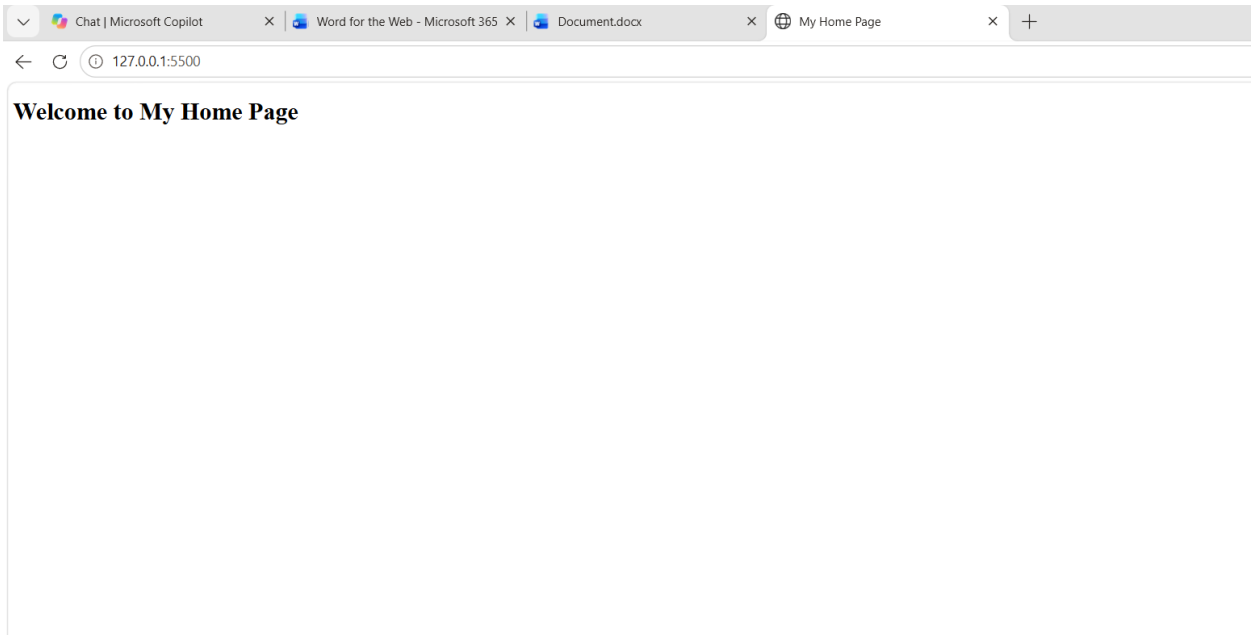
- HTML elements: h1 and p
- CSS selectors: Element selector, class selector, and ID selector
- CSS properties: `color`, `background-color`, `font-style`, `text-decoration`, and `font-size`

7.OUTPUT

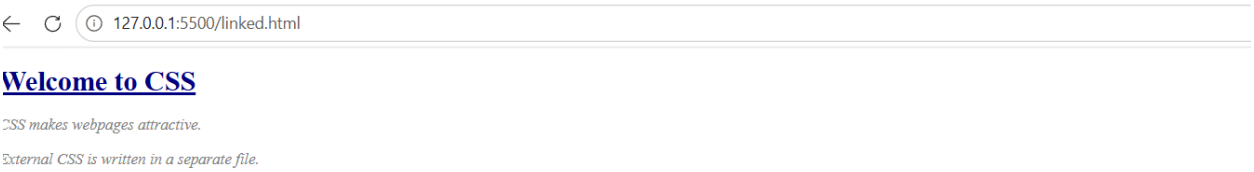
TASK1: Screenshot of `inline-internal.html` output.



TASK 2: Screenshot showing `style.css` code



TASK 3 : Screenshot of `linked.html` output.



8.RESULT

The experiment was performed successfully. Inline, internal, and external CSS were applied to HTML webpages, and different CSS selectors and text styles were used to format the content.

ASSIGNMENT 11

EXPERIMENT TITLE

Units and CSS Variables

1.AIM

To use CSS units such as `px`, `%`, and `rem`, and to store colours and spacing in CSS variables.

2.SOFTWARE/TOOLS REQUIRED

- Visual Studio Code
- HTML
- CSS
- Live Server extension
- Web browser (Google Chrome)

3.THEORY

CSS units are used to define the size of elements, text, spacing, and other properties on a webpage.

- `px` is a fixed unit used to define sizes in pixels.
- `%` is a relative unit that defines a size based on the parent element.
- `rem` is a relative unit based on the root font size.
- CSS variables are used to store values such as colours and spacing so that they can be reused throughout a webpage.

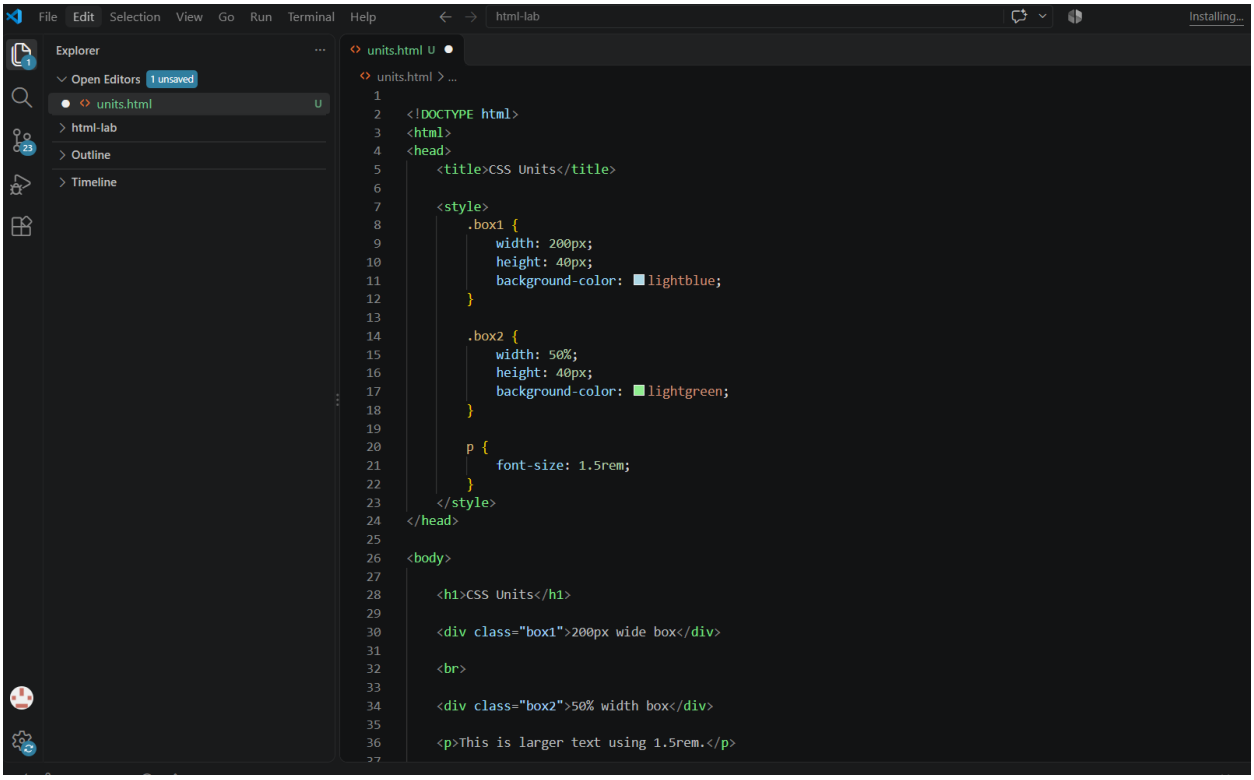
4.ALGORITHM/PROCEDURE

- Open Visual Studio Code.
- Open the `html-lab` folder.
- Create the required HTML and CSS files.
- Write the code for each task.
- Save all files.
- Open the HTML files in the browser using Live Server.
- Observe the output.

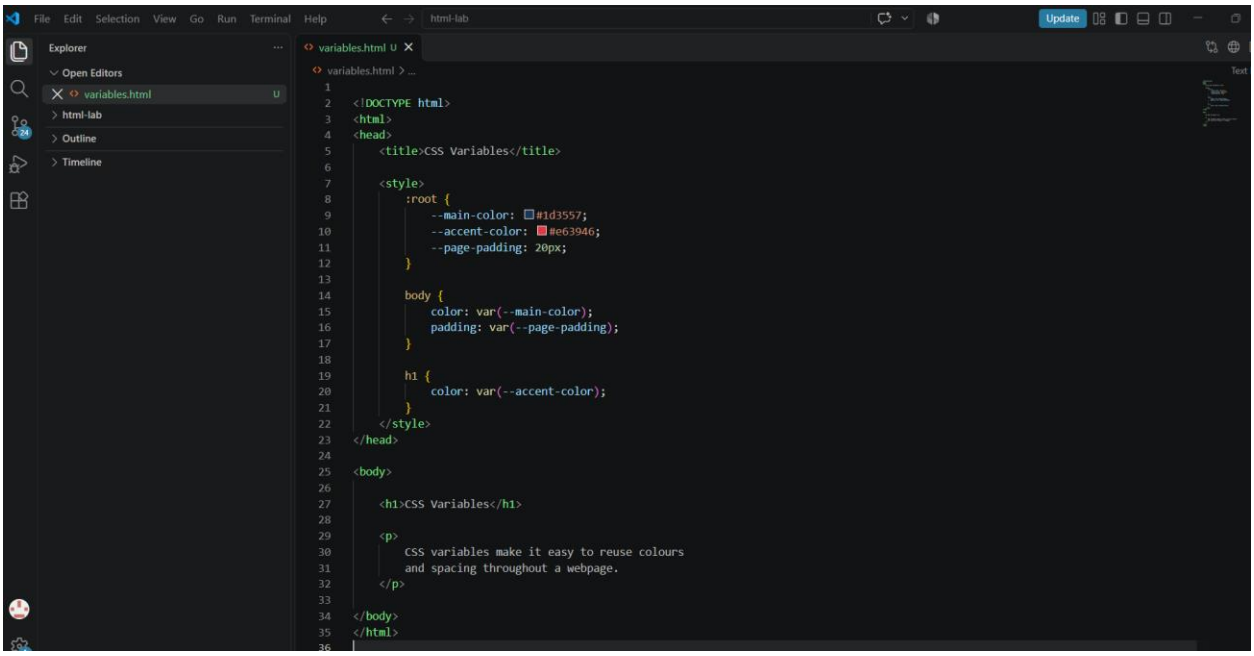
- Take screenshots of the output for all three tasks.

5.PROGRAM/QUERIES/COMMANDS

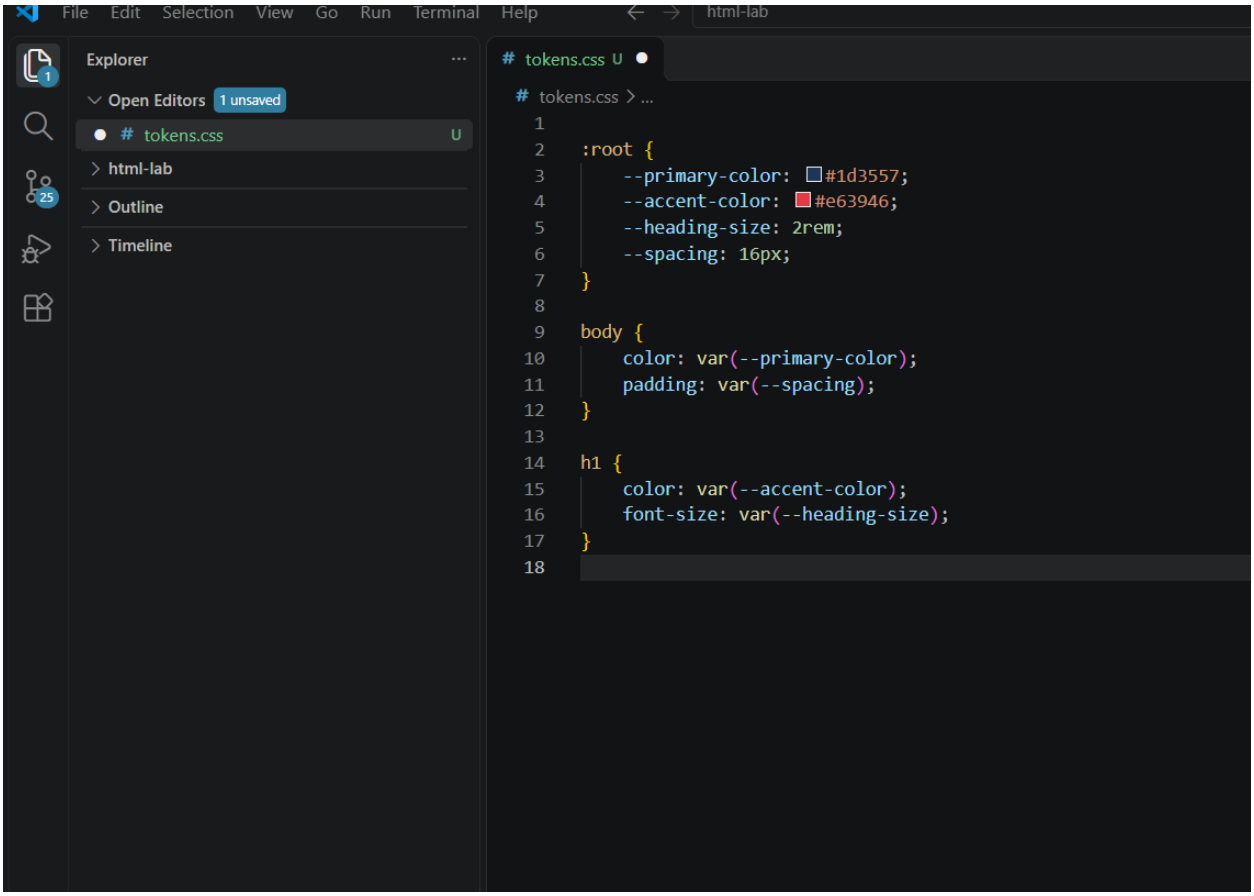
TASK 1:CREATE units.html



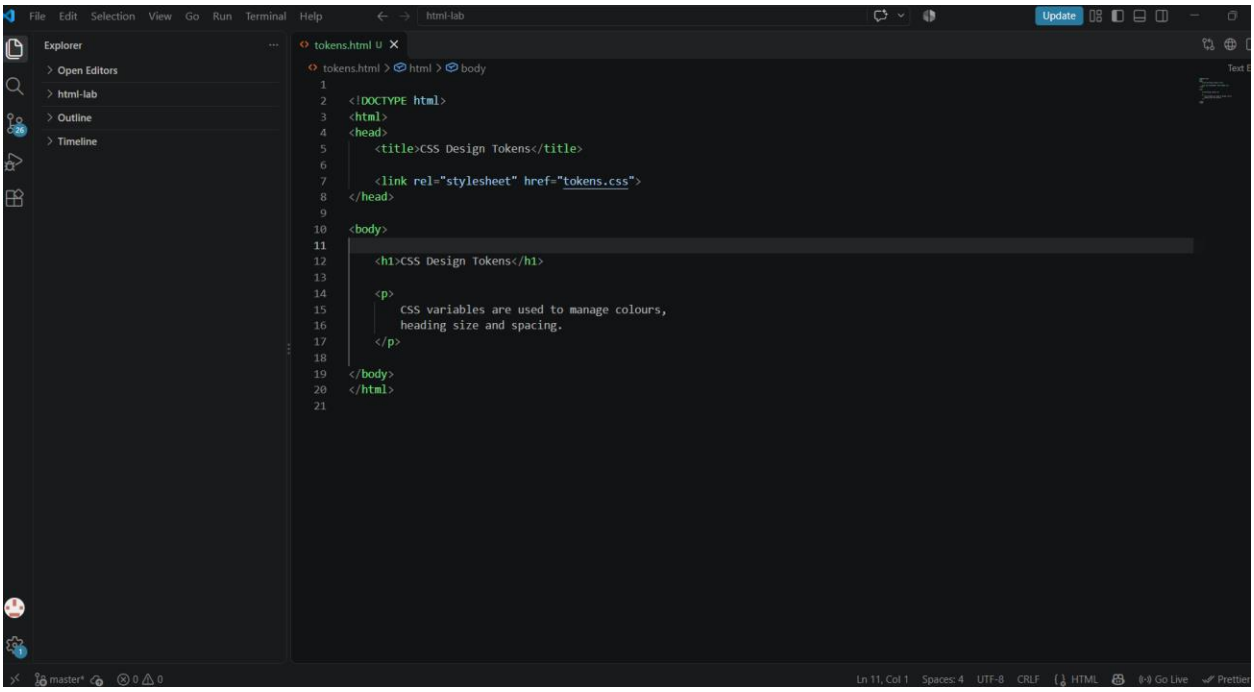
TASK 2 : CREATE variables.html



TASK 3 : CREATE tokens.css



CREATE : tokens.html



6.INPUT

7. Input

No external input is required.

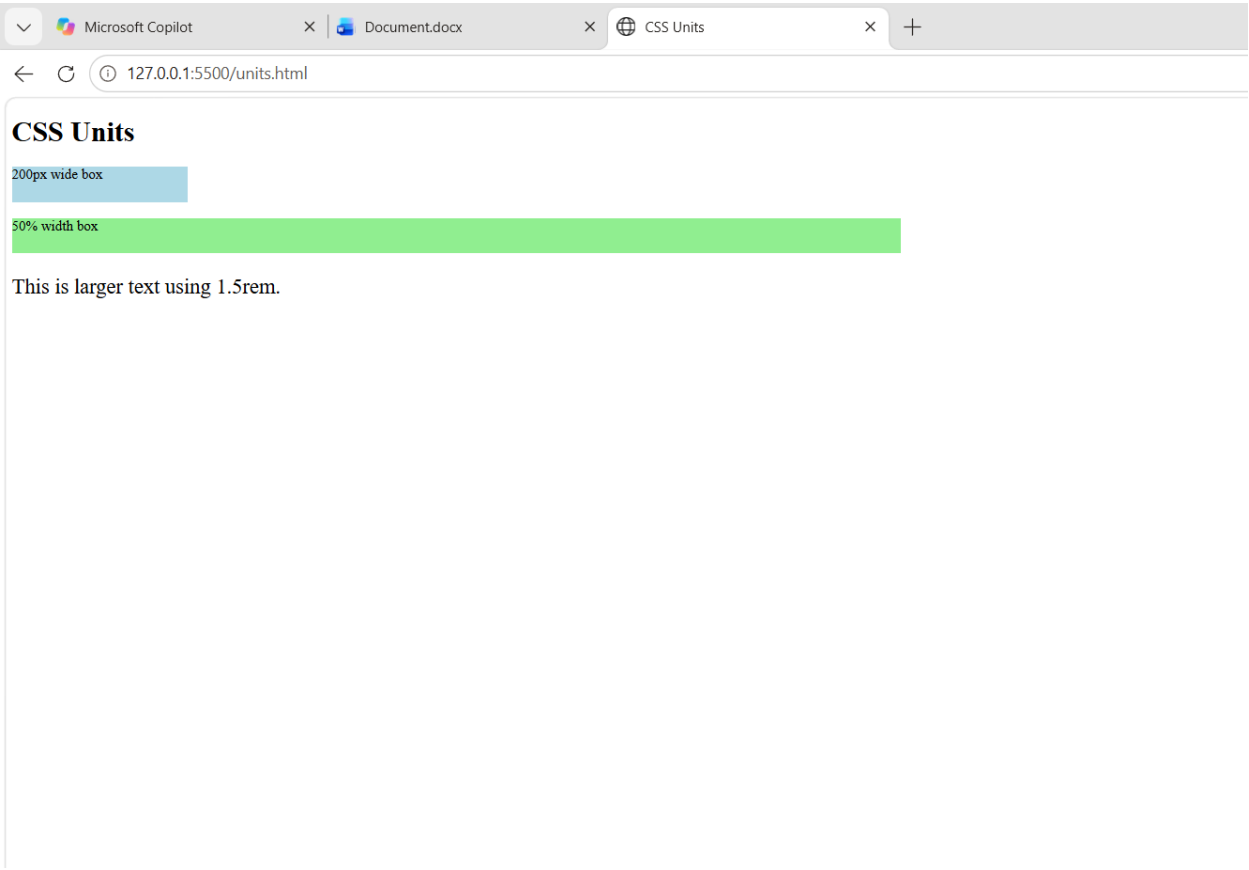
The following CSS values are used:

Property	Value
Box width	200px
Box height	40px
Half-width box	50%
Paragraph font size	1.5rem

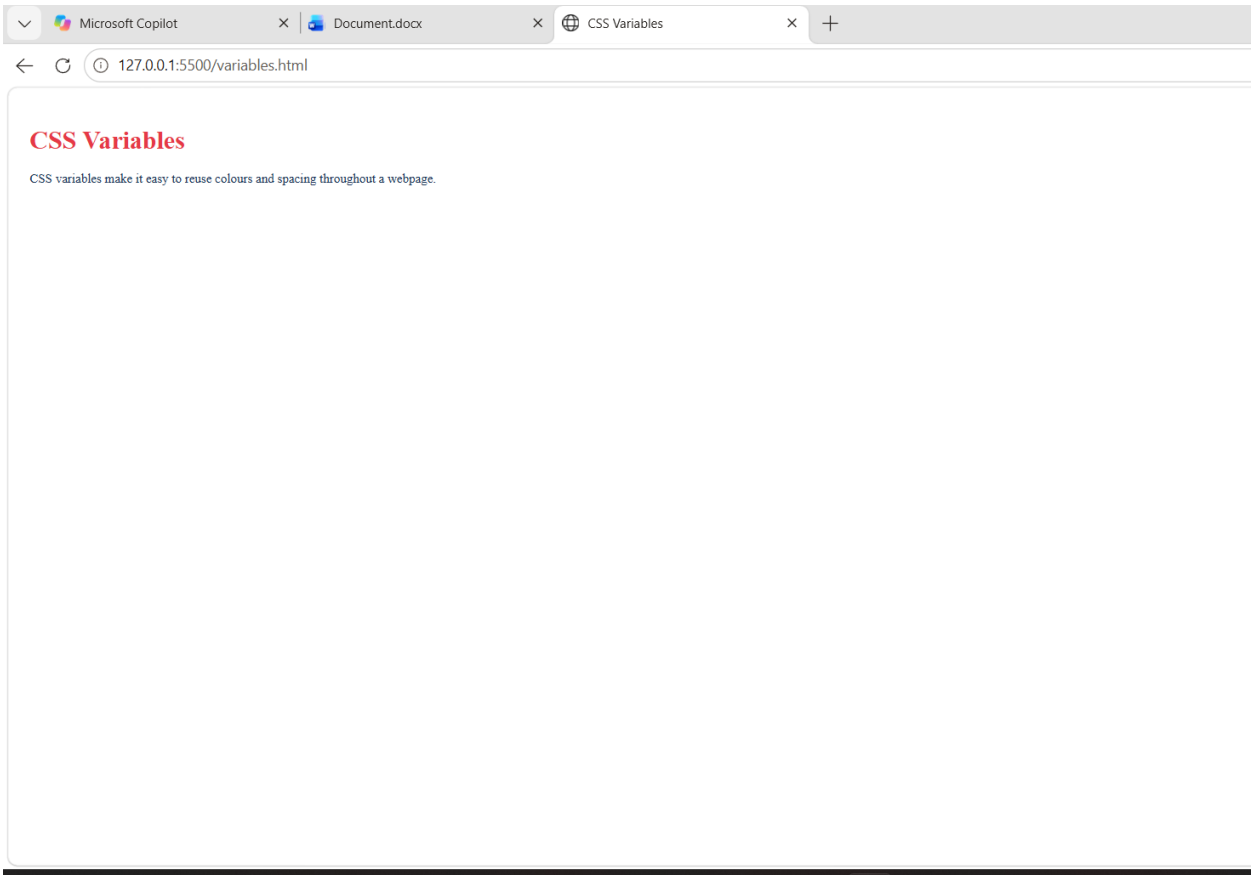
Primary colour	#1d3557
Accent colour	#e63946
Page padding	20px
Heading size	2rem

7.OUTPUT

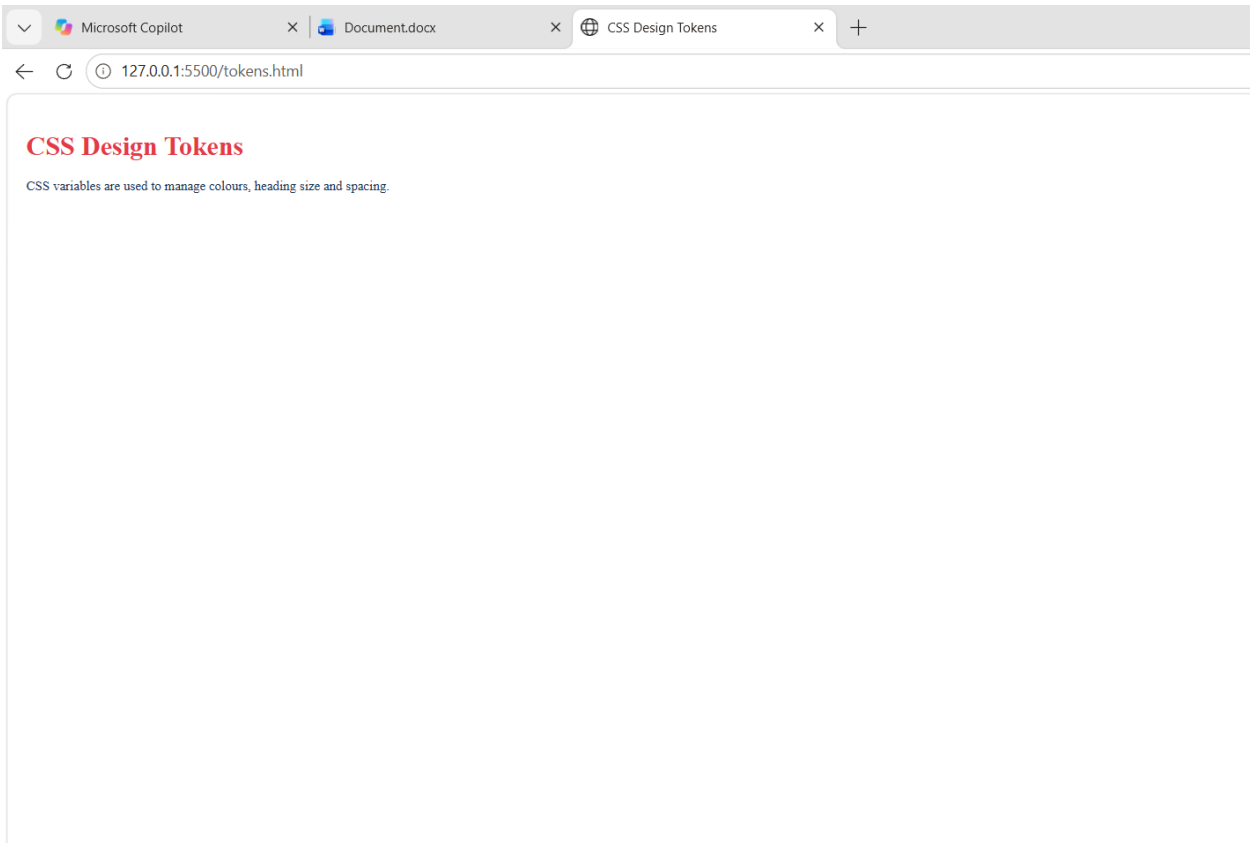
TASK1:OUTPUT



TASK 2 : OUTPUT



TASK 3 : OUTPUT



8.RESULT

The experiment was performed successfully. CSS units such as px, %, and rem were used to define element sizes and text sizes. CSS variables were also created and used to manage colours, spacing, and heading sizes.

ASSIGNMENT 12

EXPERIMENT TITLE

Study of CSS Box Model, Cards and Scrolling

1.AIM

To compare different box-sizing types, design a card using CSS properties, and create a scrollable box using the overflow property.

2.SOFTWARE/TOOLS/REQUIRED

- Visual Studio Code
- HTML
- CSS
- Live Server Extension
- Web Browser (Google Chrome)

3.THEORY

The CSS Box Model describes how elements are displayed on a webpage. It consists of content, padding, border, and margin.

- Content-box: Width and height apply only to the content. Padding and borders are added separately.
- Border-box: Width and height include content, padding, and border.
- Border-radius: Used to create rounded corners.
- Padding: Creates space inside an element.
- Overflow: Controls what happens when content exceeds the size of a box.
- Background-color: Adds a background colour to an element.

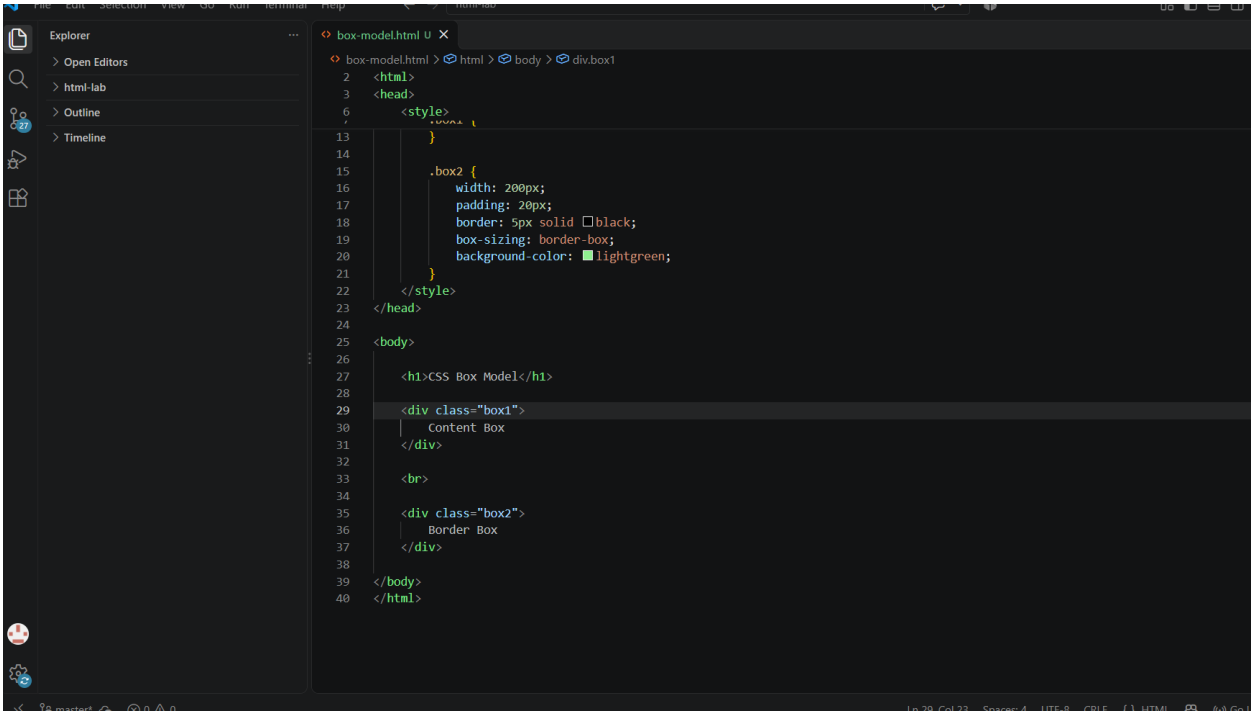
5.ALGORITHM/PROCEDURE

- Open Visual Studio Code.
- Open the `html-lab` folder.
- Create a file named `box-model.html`.
- Create two boxes using content-box and border-box properties.
- Create a file named `card.html`.

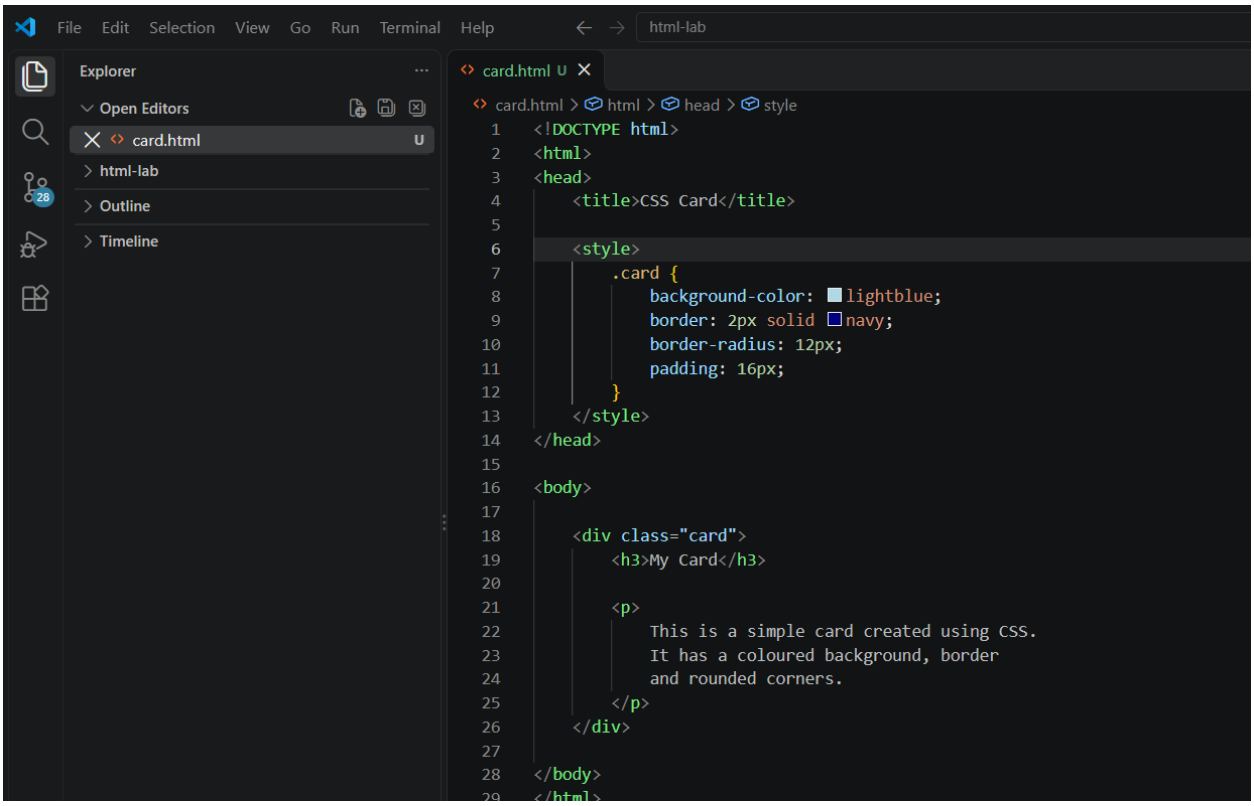
- Design a card using background colour, border, padding, and rounded corners.
- Create a file named `scroll.html`.
- Create a box with a fixed height of 120px and set overflow to auto.
- Add five paragraphs inside the box.
- Save all files.
- Open each HTML file using Live Server.
- Observe the output and take screenshots.

6.PROGRAM/QUERIES/COMMANDS

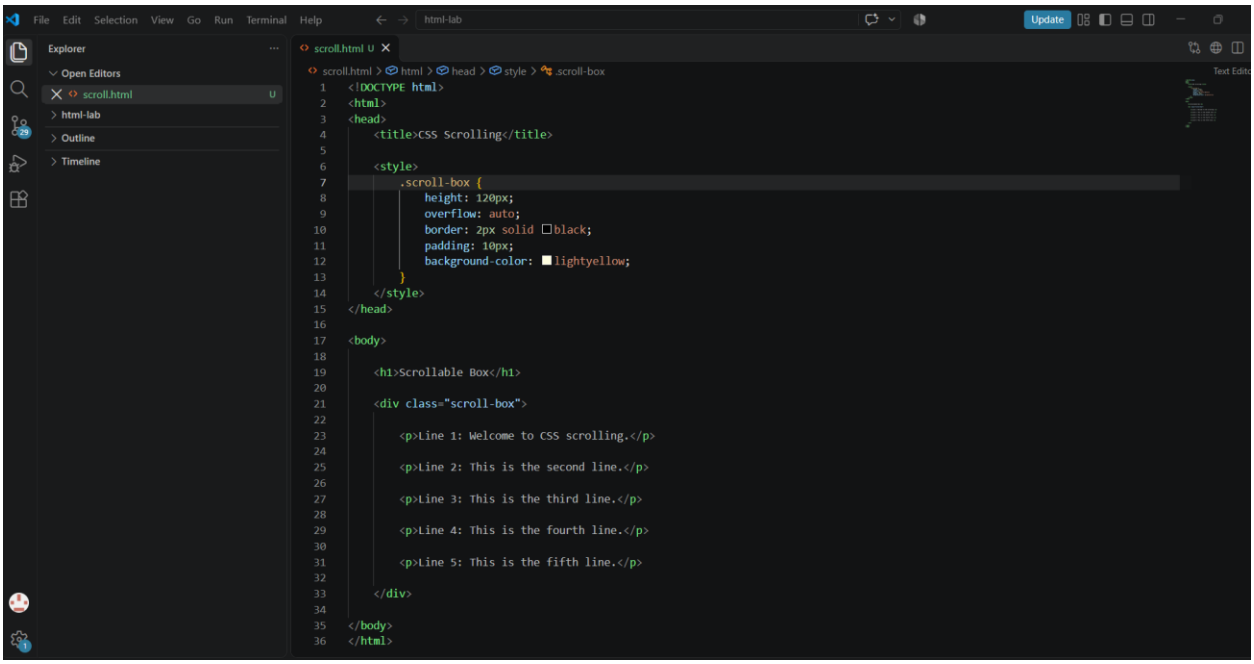
TASK 1 : create box-model.html



TASK 2: create card.html



TASK 3 : create scroll.html



7.INPUT

TASK1:

- Width: 200px
- Padding: 20px
- Border: 5px
- Box sizing: content-box and border-box

TASK 2:

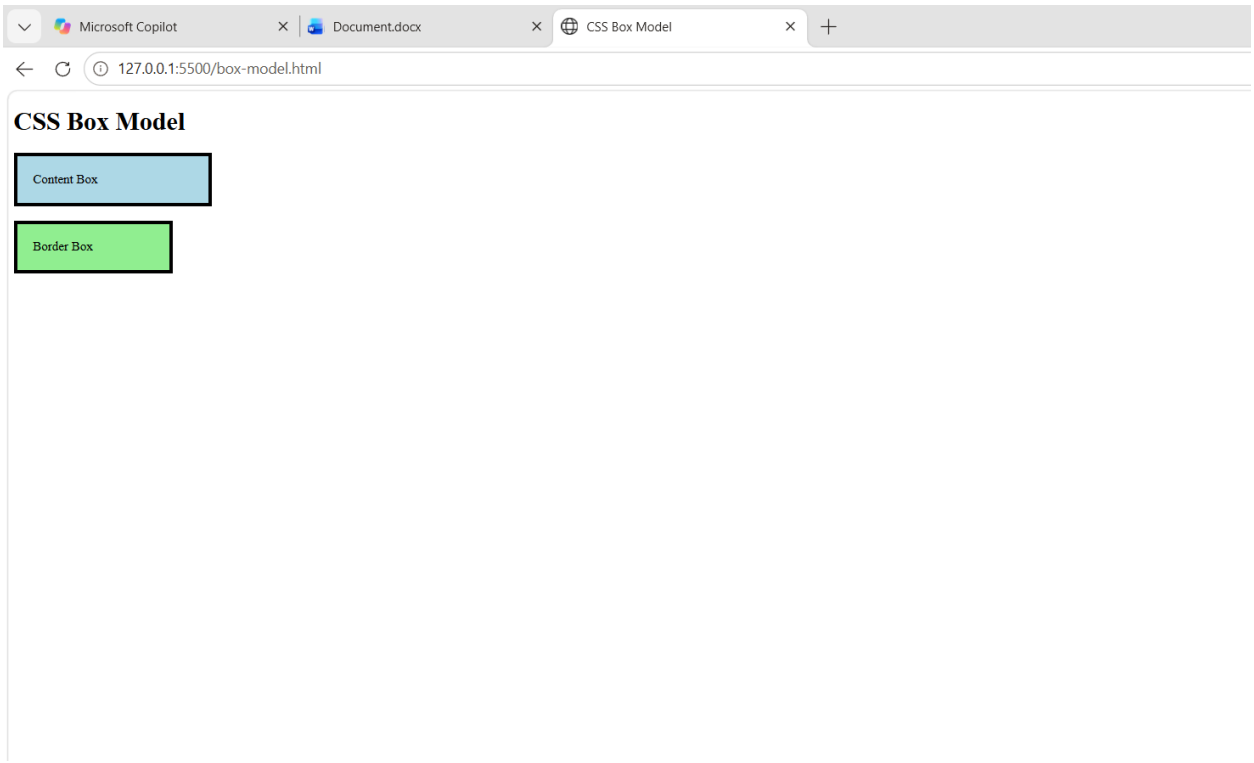
- Background colour: Light blue
- Border: 2px solid navy
- Border radius: 12px
- Padding: 16px

TASK 3:

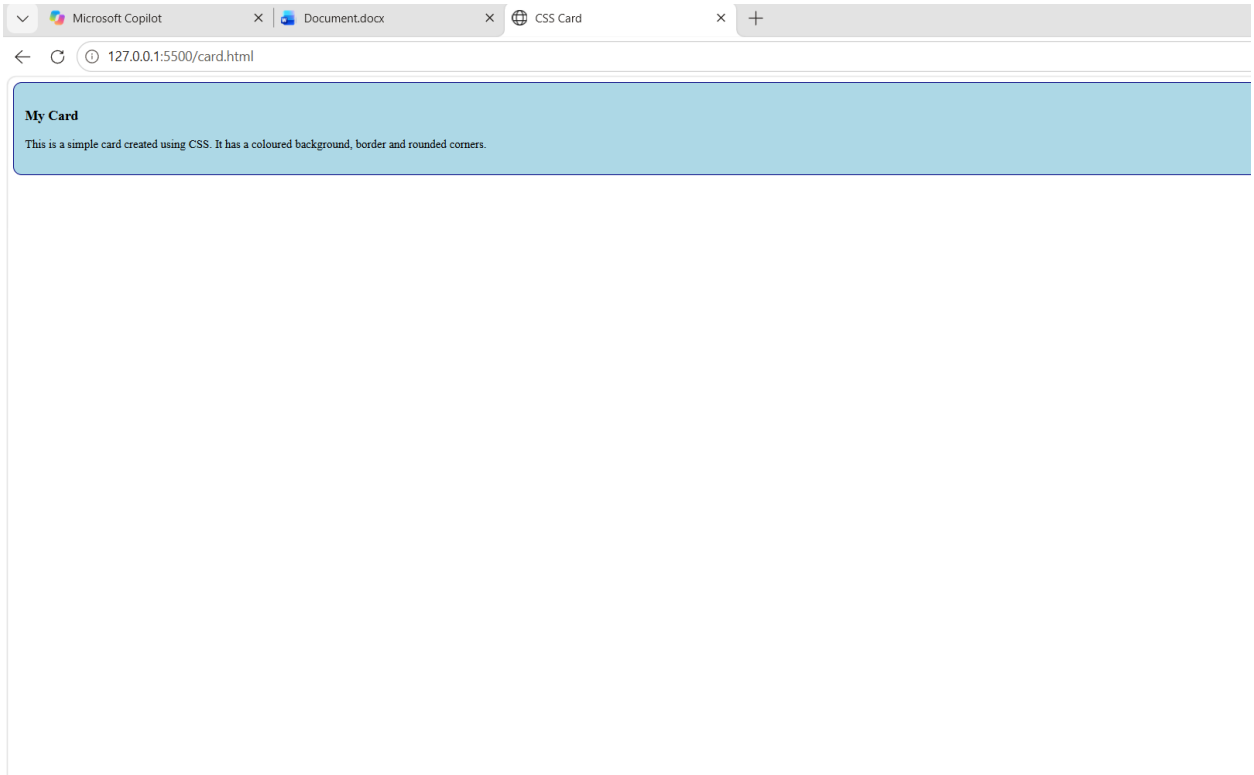
- Height: 120px
- Overflow: auto
- Number of paragraphs: 5
- Text: Line 1 to Line 5

7.OUTPUT

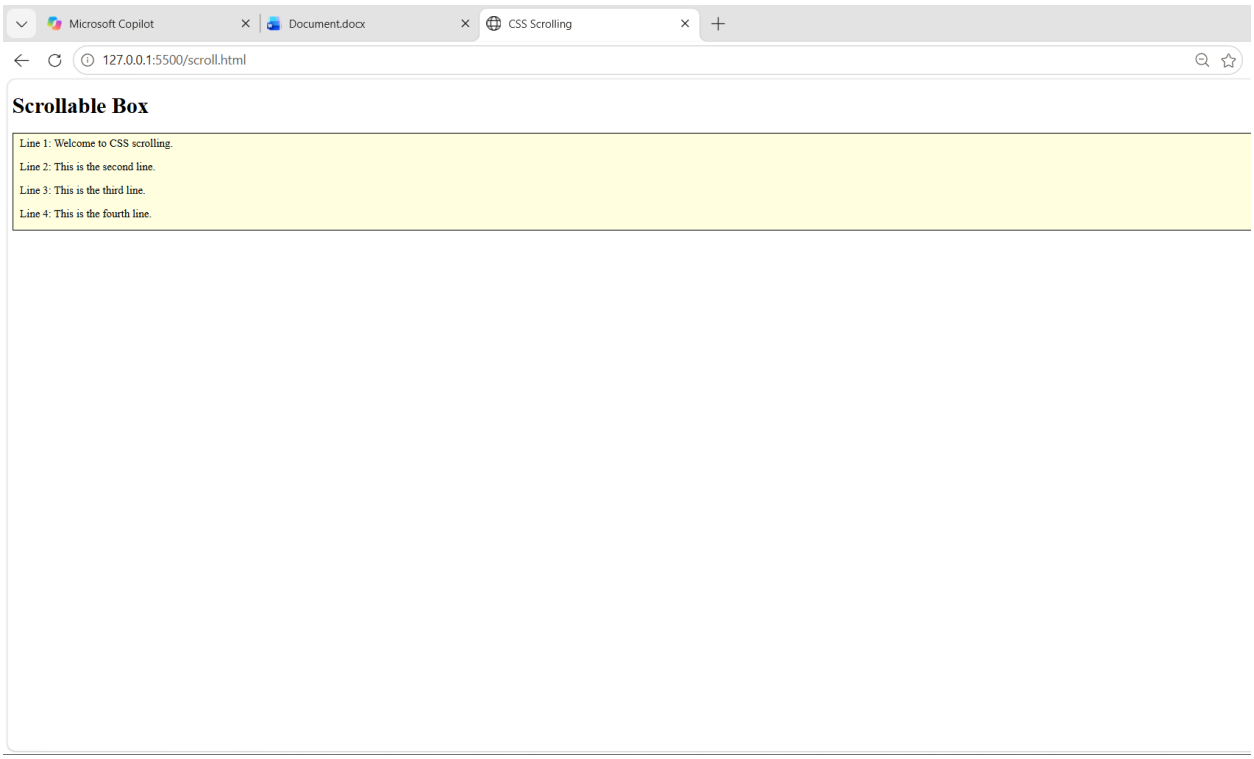
TASK 1 : Screenshot of `box-model.html` output.



TASK 2 : Screenshot of `card.html` output.



TASK 3 : Screenshot of `scroll.html` output



8.RESULT

The experiment was performed successfully. Different CSS box-sizing properties were compared, a styled card was created, and a scrollable box was implemented using the CSS overflow property.

ASSIGNMENT 13

EXPERIMENT TITLE

Creating Webpage Layouts Using CSS Flexbox

1.AIM

To create a navigation bar, centred cards, and flexible card layouts using CSS Flexbox.

2.SOFTWARE/TOOLS REQUIRED

- Visual Studio Code
- HTML
- CSS
- Live Server Extension
- Web Browser (Google Chrome)

3.THEORY

CSS Flexbox is a layout model used to arrange HTML elements in rows or columns.

Important Flexbox properties:

- `display: flex`: Enables Flexbox layout.
- `justify-content`: Aligns items horizontally.
- `align-items`: Aligns items vertically.
- `flex-direction`: Defines the direction of items.
- `flex-wrap`: Allows items to move to the next row.
- `gap`: Adds space between items

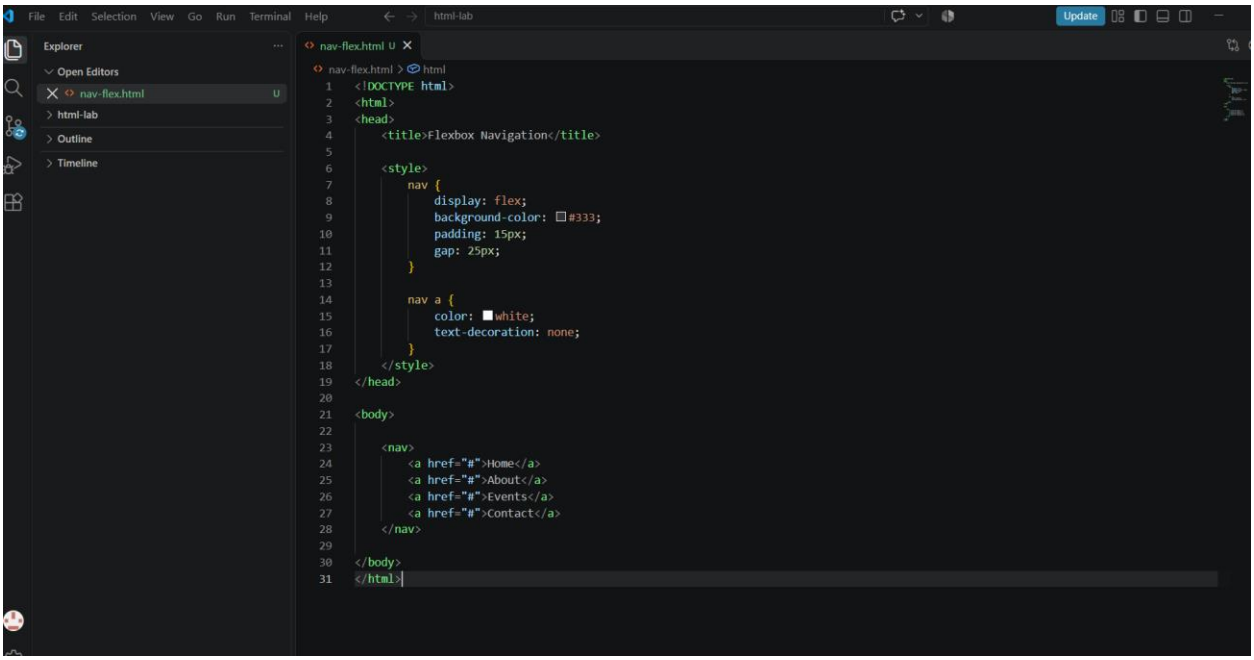
4.ALGORITHM/PROCEDURE

- Open Visual Studio Code.
- Open the `html-lab` folder.
- Create a file named `nav-flex.html`.
- Create a horizontal navigation bar with four links.
- Create a file named `centred-cards.html`.
- Create three cards with headings and prices.

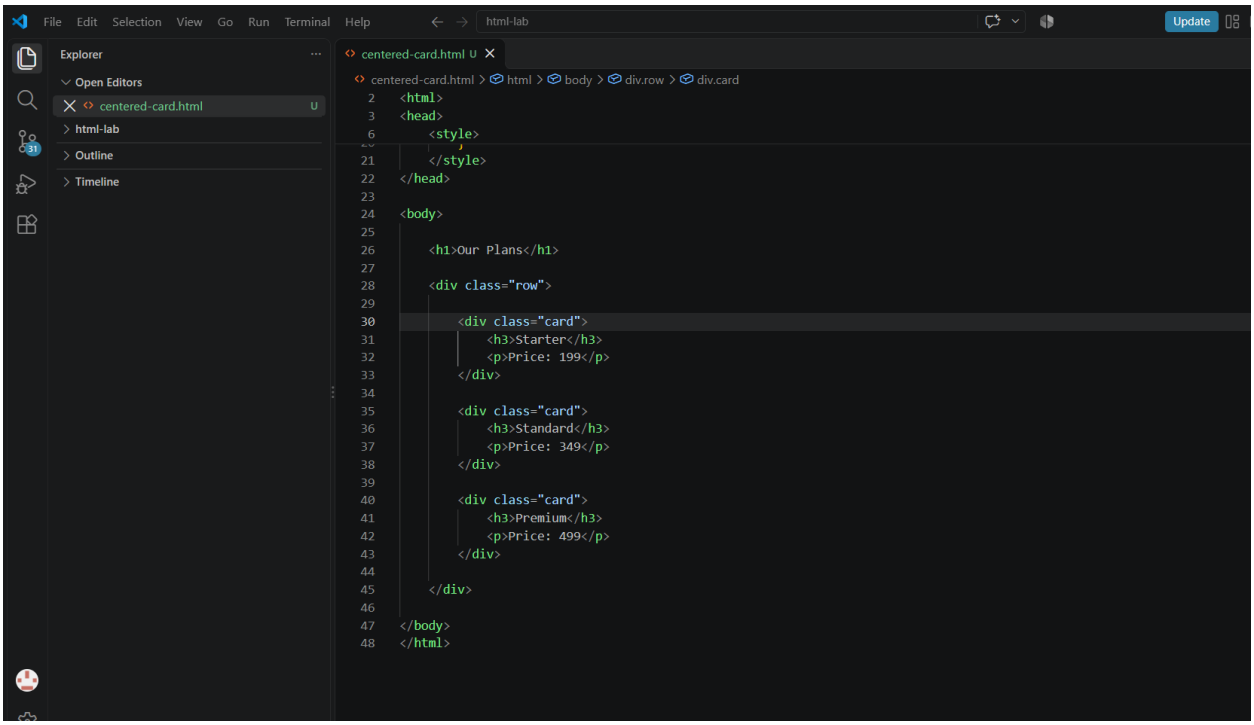
- Use Flexbox properties to centre the cards.
- Save all files.
- Open each HTML file using Live Server.
- Observe the output and take screenshots.

5.PROGRAM/QUERIES/COMMANDS

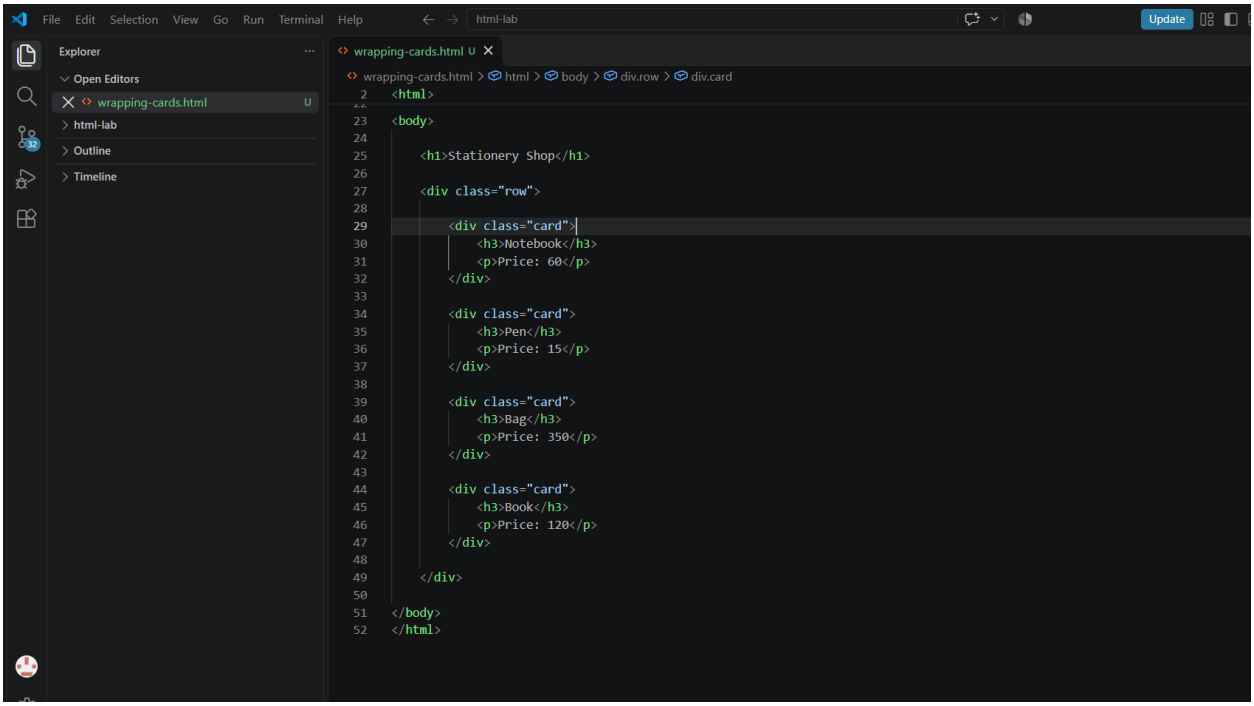
TASK 1:create nav-flex.html



TASK 2 : CREATE centred-card.html



TASK 3 : create wrapping-cards.html



6.INPUT

Task 1:

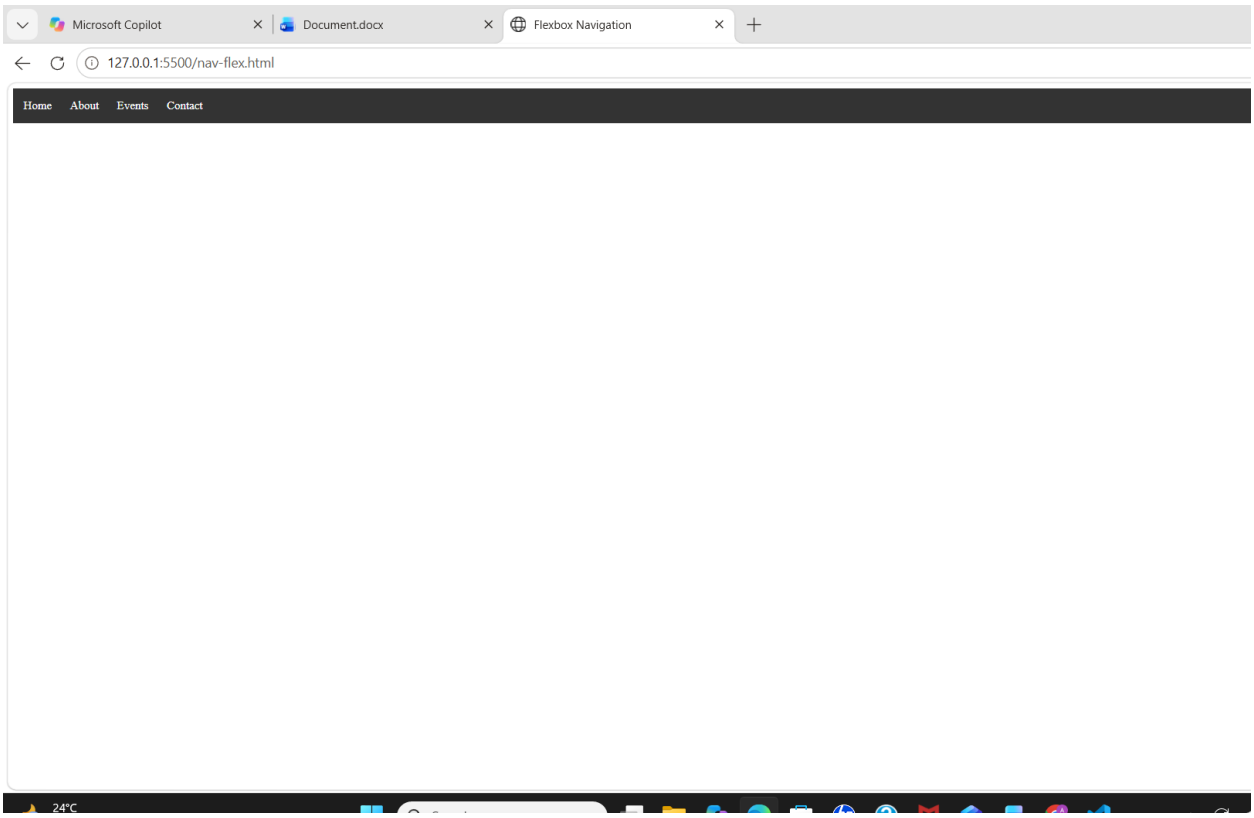
- Navigation links: Home, About, Events, Contact.
- Background colour: Dark grey.
- Text colour: White.

Task 2:

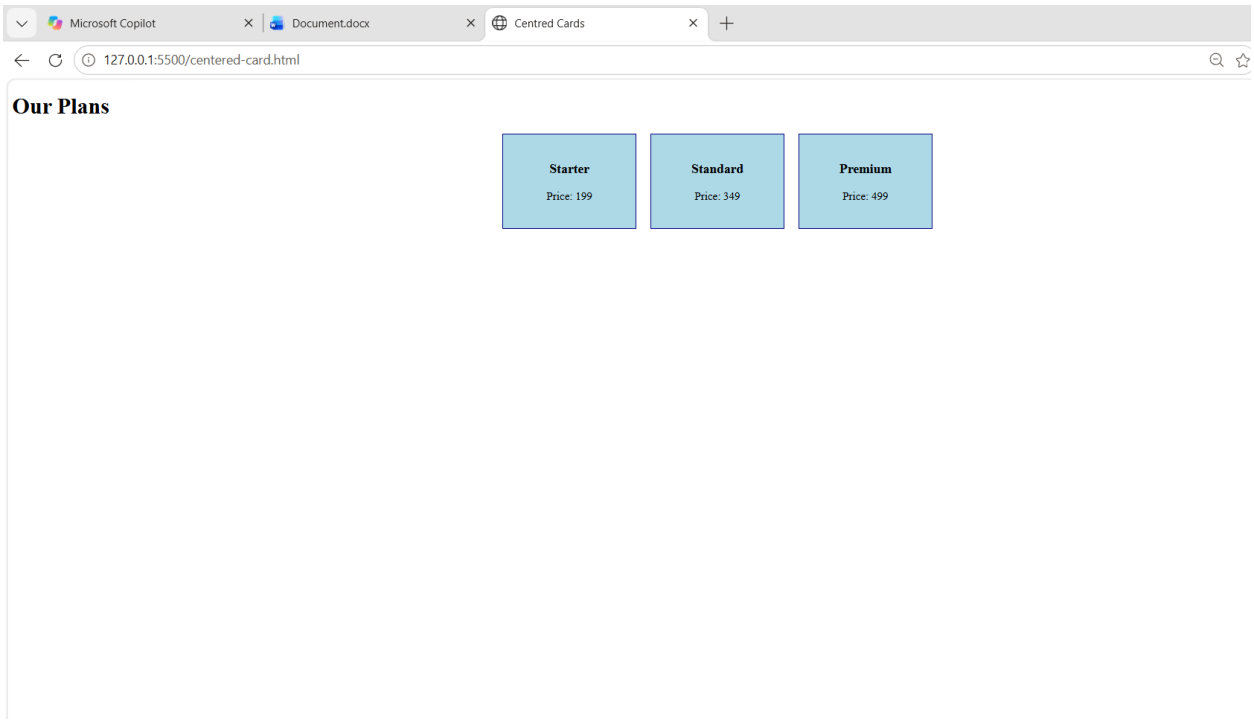
- Starter price: 199
- Standard price: 349
- Premium price: 499
- Flexbox properties: `justify-content` and `align-items`.

7.OUTPUT

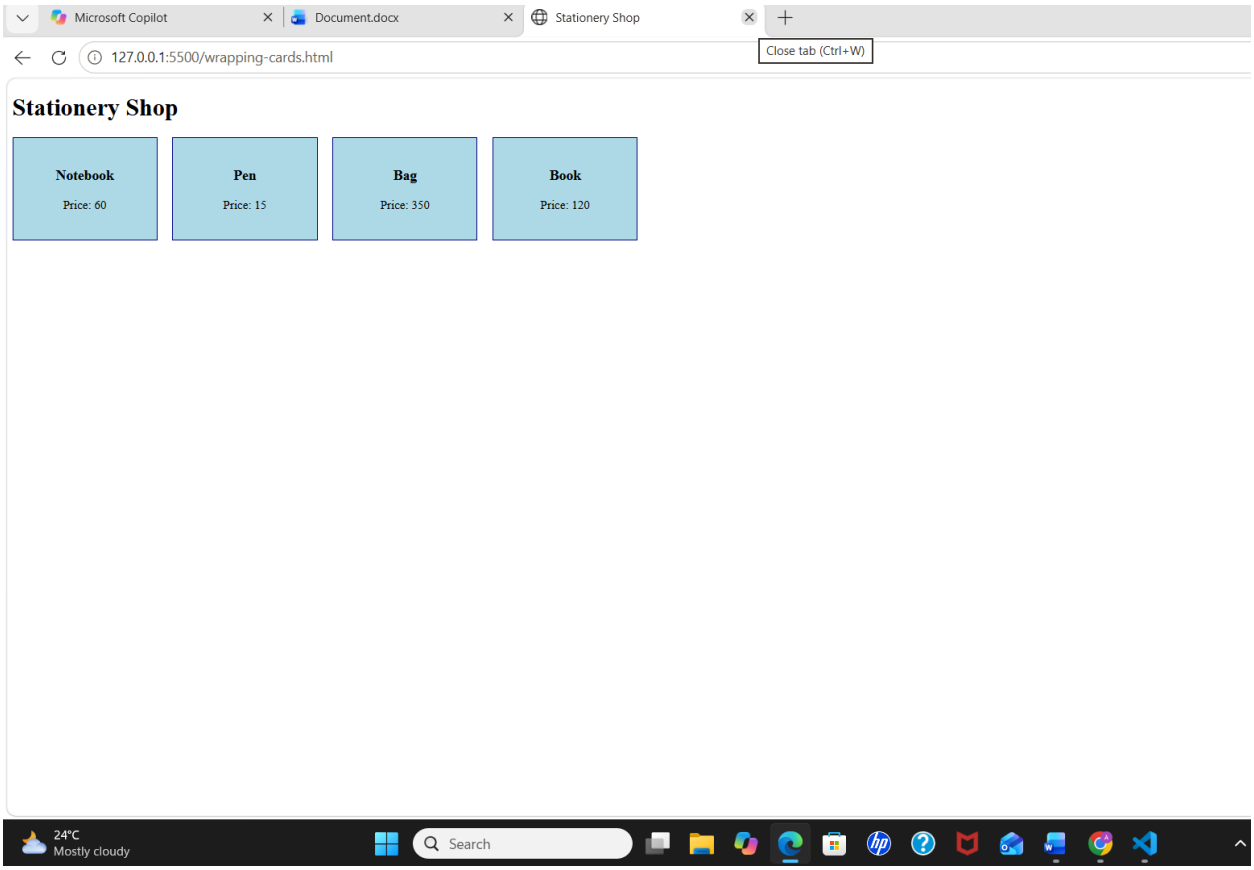
TASK 1 :screenshot of nav-flex.html



TASK 2 : screenshot of centred-card.html



TASK 3 : screenshot of wrapping.html



8.RESULT

The experiment was performed successfully. CSS Flexbox was used to create a horizontal navigation bar, centre multiple cards, and arrange stationery cards in wrapping rows using the `flex-wrap` property

ASSIGNMENT 14

EXPERIMENT TITLE

Creating Webpage Layouts Using CSS Grid

1.AIM

To create a grid with equal columns, a responsive gallery, and a webpage layout using named grid areas.

2.SOFTWARE/TOOLS REQUIRED

- Visual Studio Code
- HTML
- CSS
- Live Server Extension
- Web Browser (Google Chrome)

3.THEORY

CSS Grid is a layout system used to arrange webpage elements in rows and columns.

Important CSS Grid properties:

- `display: grid`: Enables the CSS Grid layout.
- `grid-template-columns`: Defines the number and size of columns.
- `gap`: Adds space between grid items.
- `fr`: Represents a fraction of the available space.
- `repeat()`: Repeats column or row definitions.
- `minmax()`: Defines minimum and maximum sizes.
- `grid-template-areas`: Defines named areas for webpage layouts.
- `auto-fill`: Automatically fits as many columns as possible.

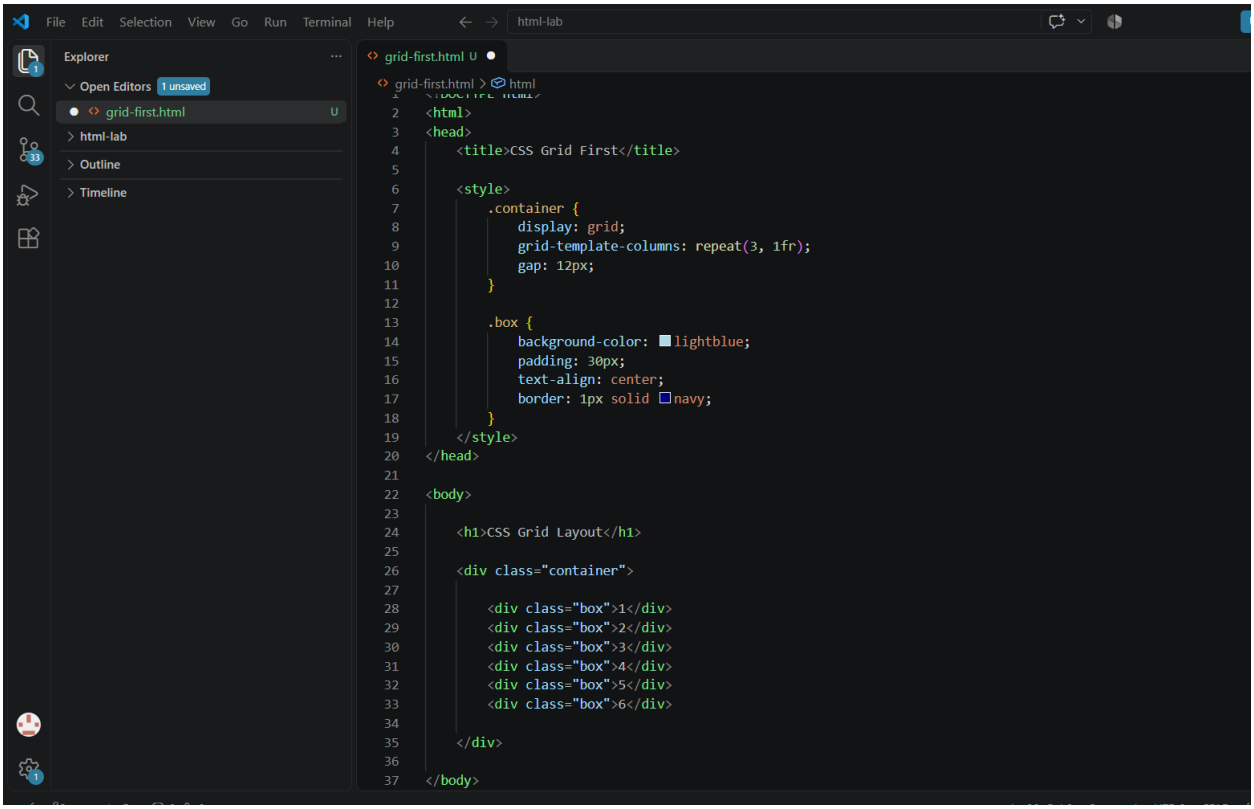
4.ALGORITHM/PROCEDURE

- Open Visual Studio Code.
- Open the `html-lab` folder.
- Create a file named `grid-first.html`.
- Create six boxes using CSS Grid with three equal columns.

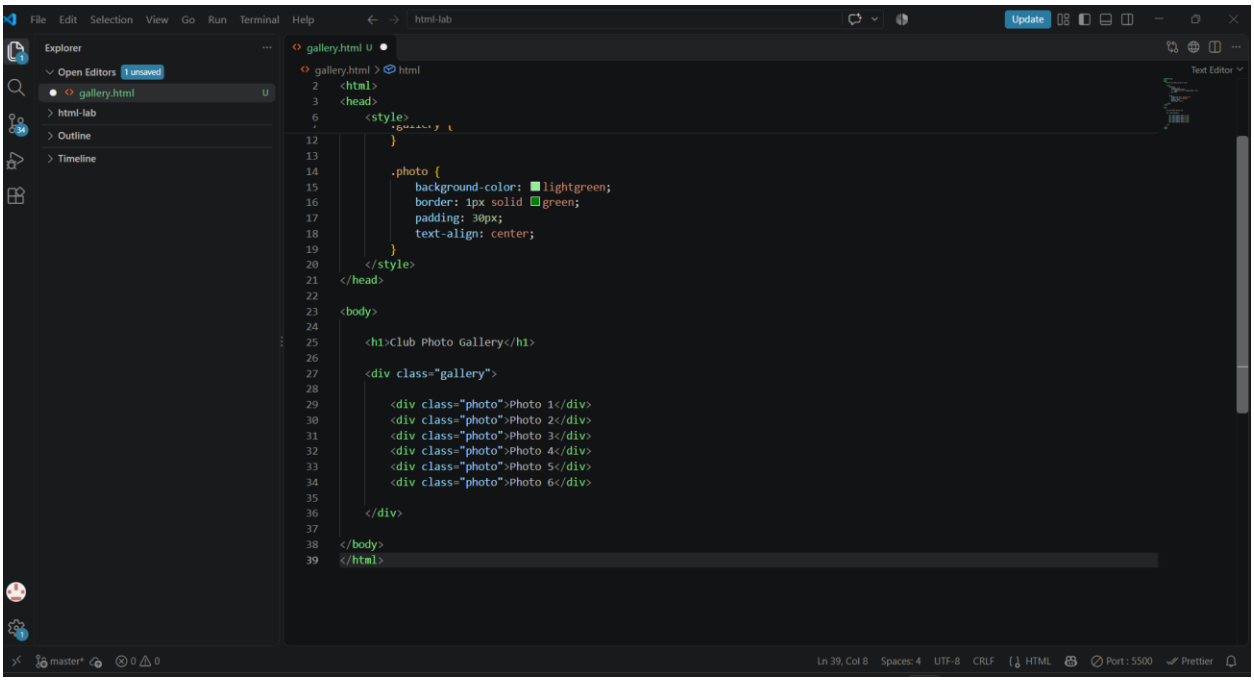
- Create a file named `gallery.html`.
- Create a photo gallery with six boxes labelled Photo 1 to Photo 6.
- Use responsive grid properties to adjust the columns.
- Create a file named `dashboard-areas.html`.
- Create a dashboard layout with a header, sidebar, main area, and footer.
- Use `grid-template-areas` to arrange the sections.
- Save all files.
- Open each HTML file using Live Server.
- Observe the output and take screenshots.

5.PROGRAM/QUERIES/COMMANDS

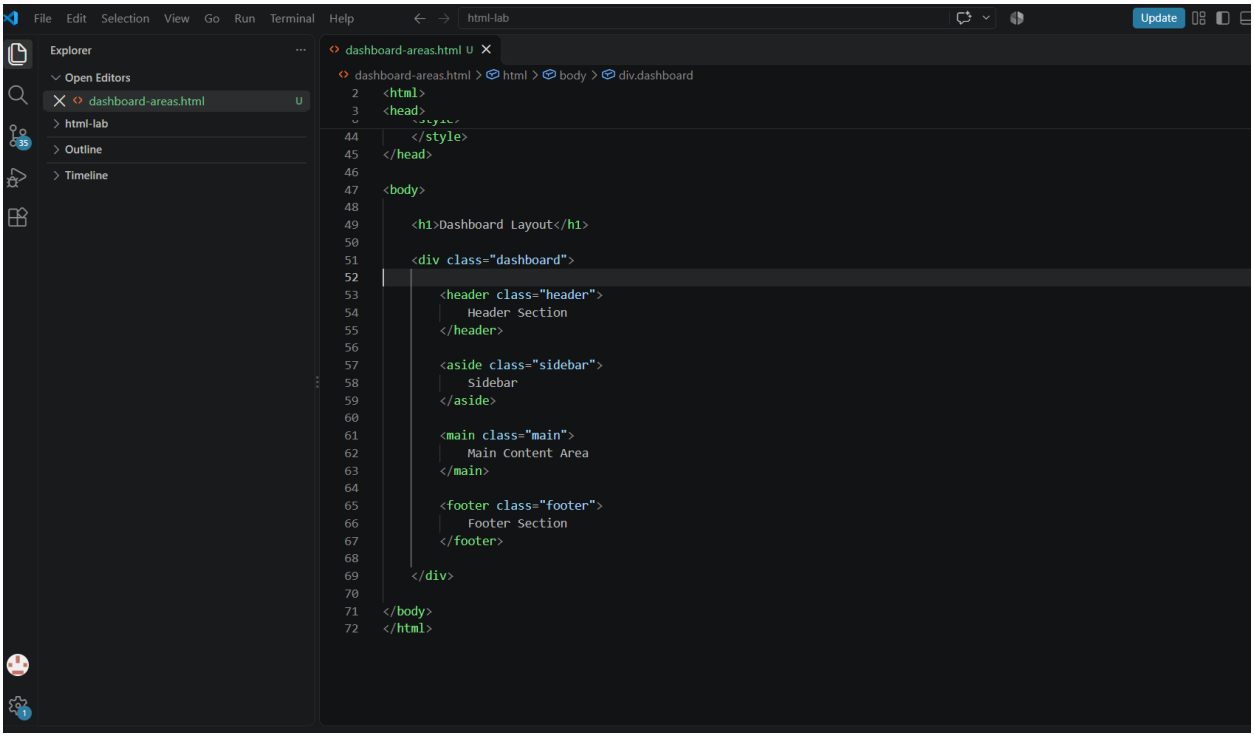
TASK 1 : CREATE grid-first.html



TASK 2 : CREATE gallery.html



TASK 3 : CREATE dashboard -areas.html



6.INPUT

Task 1:

- Number of boxes: 6

- Number of columns: 3
- Gap: 12px
- Grid unit: 1fr

Task 2:

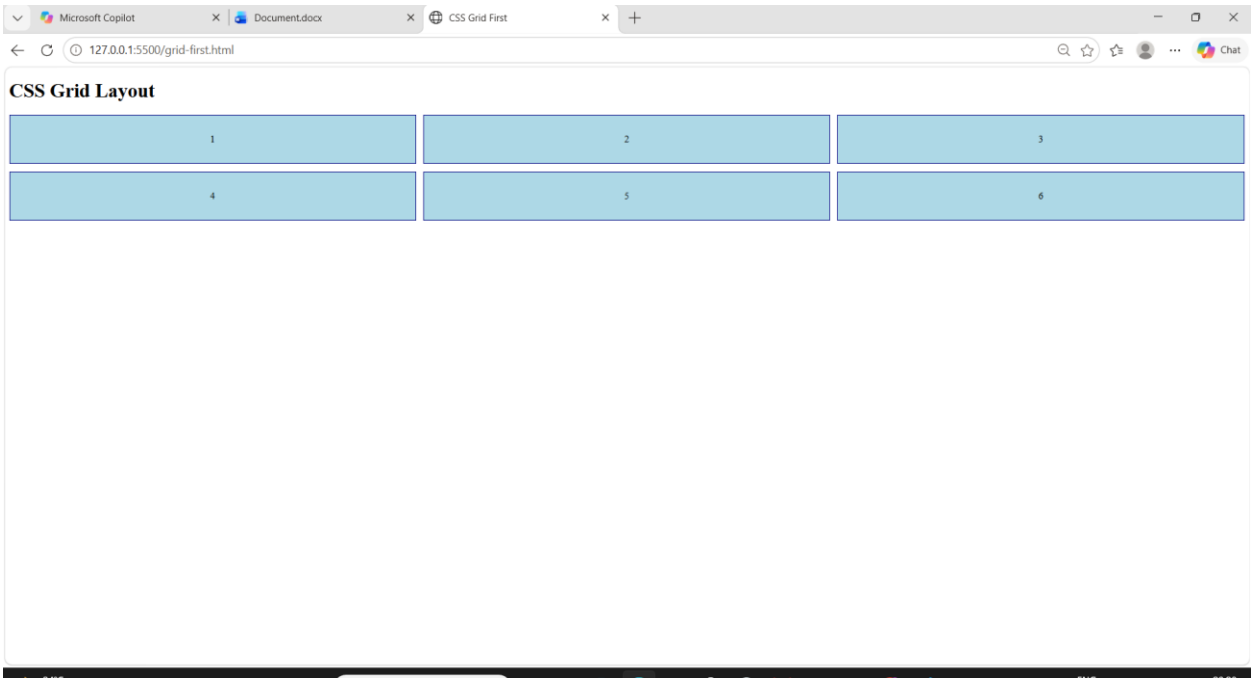
- Heading: Club Photo Gallery
- Gallery items: Photo 1, Photo 2, Photo 3, Photo 4, Photo 5, Photo 6
- Minimum column width: 150px
- Maximum column width: 1fr
- Gap: 10px

Task 3:

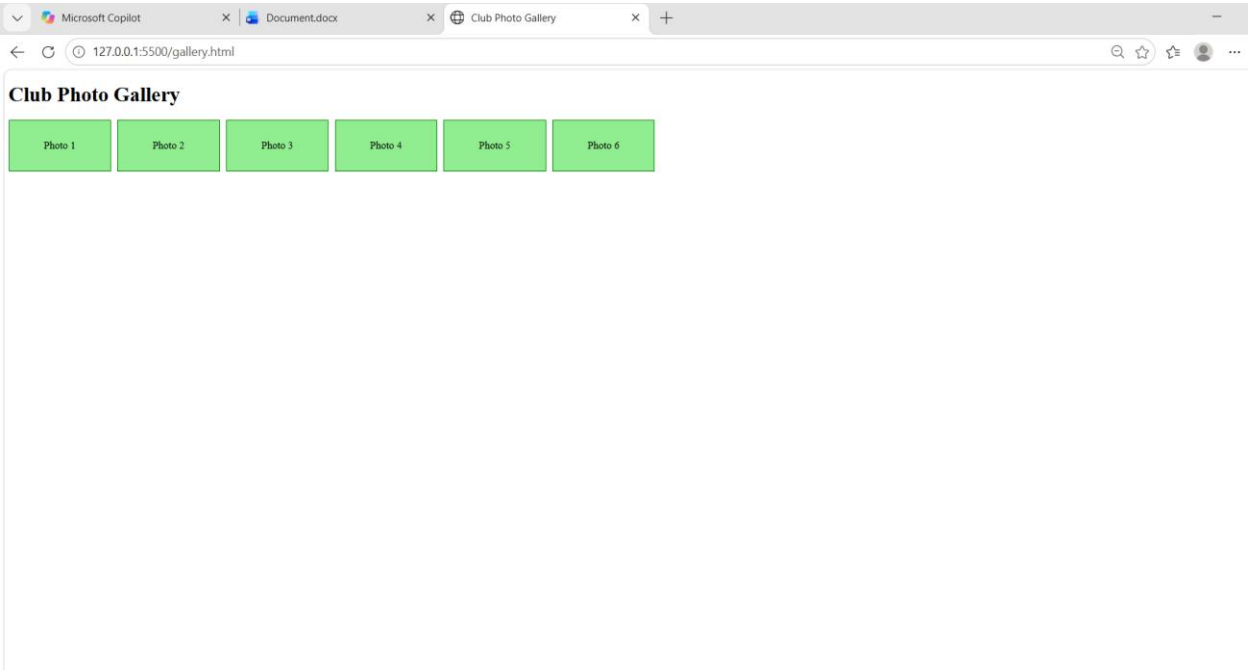
- Header: Header Section
- Sidebar: Sidebar
- Main area: Main Content Area
- Footer: Footer Section
- Grid columns: 200px and 1fr

7.OUTPUT

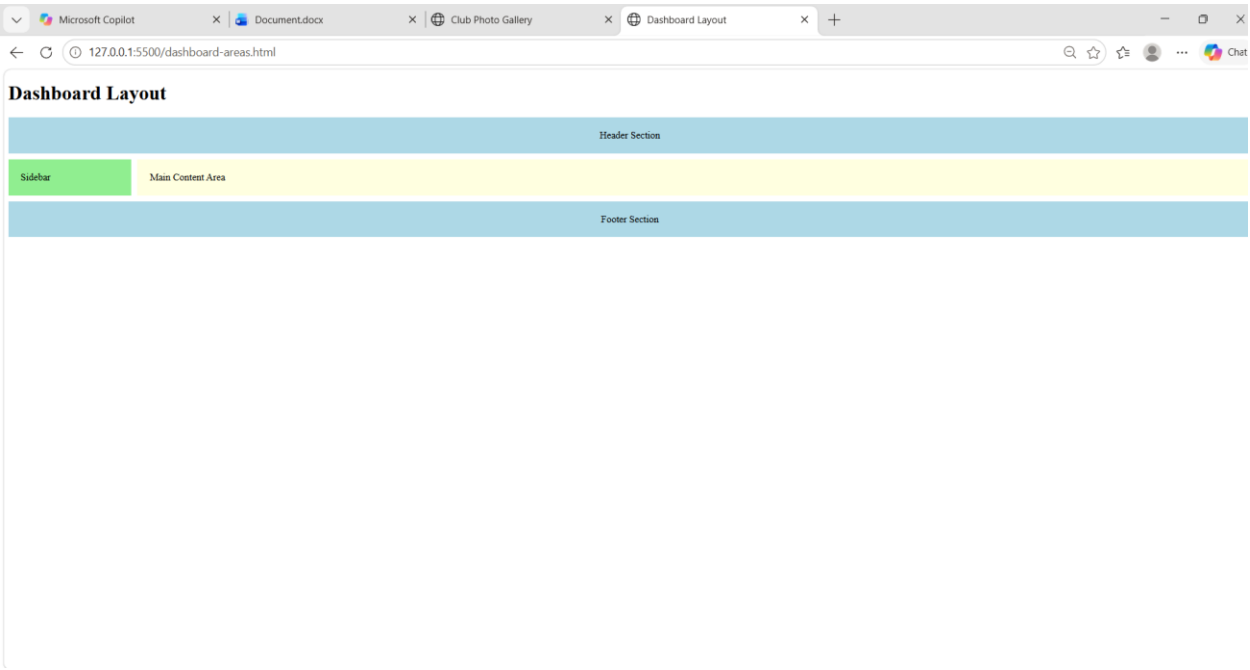
TASK 1: Screenshot of `grid-first.html` output.



TASK 2 : Screenshot of `gallery.html` output.



TASK 3 : Screenshot of `dashboard-areas.html` output



8.RESULT

The experiment was performed successfully. CSS Grid was used to create equal columns, a responsive photo gallery, and a dashboard layout using named grid areas.

ASSIGNMENT 15

EXPERIMENT TITLE

Creating Responsive Webpages Using CSS Media Queries

1.AIM

To use media queries to change webpage layouts, make images responsive, and adjust text sizes using CSS units such as `rem`.

2.SOFTWARE/TOOLS REQUIRED

- Visual Studio Code
- HTML
- CSS
- Live Server Extension
- Web Browser (Google Chrome)

3.THEORY

Responsive web design is used to make webpages look good on different screen sizes, including mobile phones, tablets, and computers.

Important CSS properties:

- Media Queries: Apply different CSS styles depending on screen size.
- Viewport Meta Tag: Helps webpages adjust to the device width.
- `max-width: 100%`: Prevents an image from becoming wider than its container.
- `height: auto`: Maintains the original image proportions.
- `rem`: A relative unit used to define font sizes.
- `@media`: Used to apply CSS rules at specific screen widths.

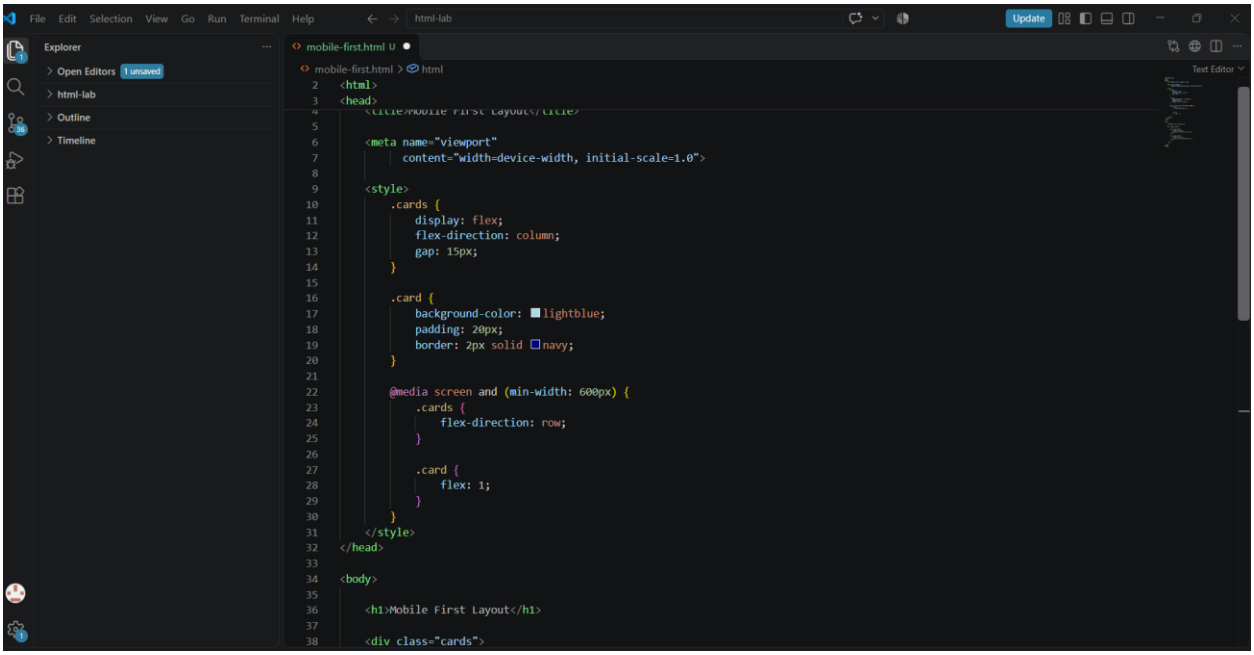
4.ALGORITHM/PROCEDURE

- Open Visual Studio Code.
- Open the `html-lab` folder.
- Create a file named `mobile-first.html`.
- Create two cards with headings and paragraphs.

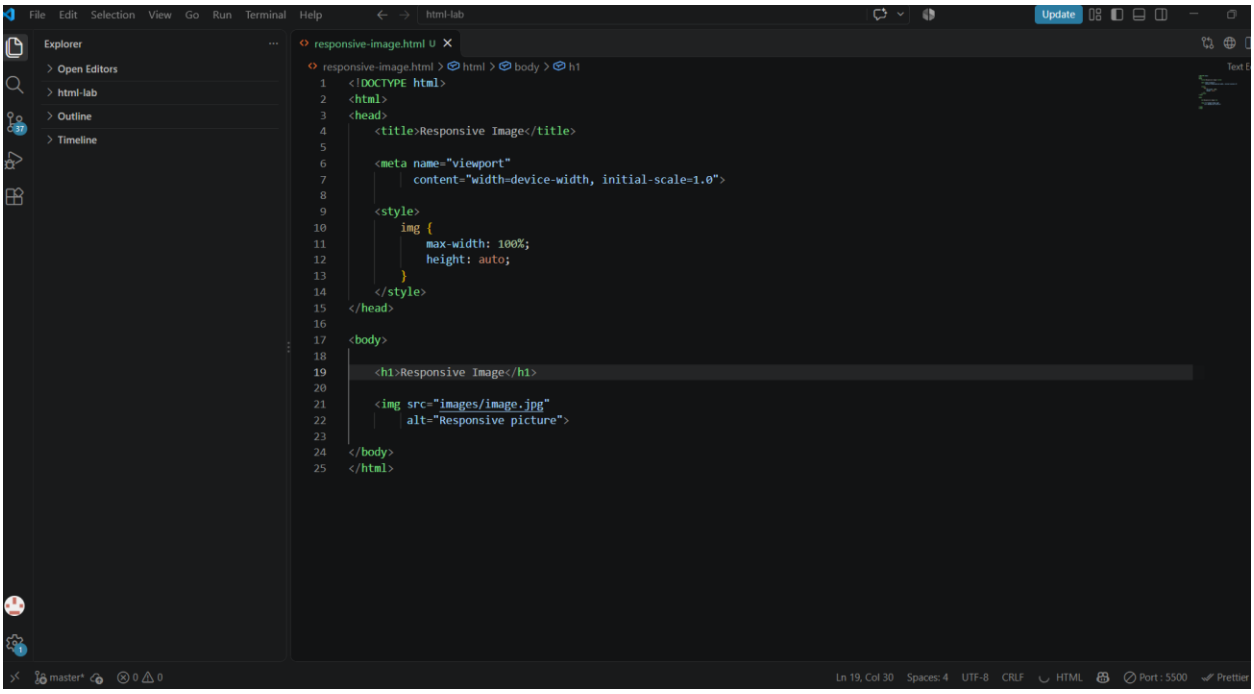
- Apply a media query to display the cards side by side on screens 600px wide and above.
- Create a file named `responsive-image.html`.
- Add an image from the images folder and make it responsive.
- Create a file named `responsive-text.html`.
- Add an h1 heading and a paragraph.
- Set the body font size to 1rem and the heading font size to 1.5rem.
- Add a media query to increase the heading size to 2.5rem on screens 600px wide and above.
- Save all files.
- Open each HTML file using Live Server.
- Resize the browser window to check the responsive layouts.
- Take screenshots of the outputs.

5.PROGRAM/QUERIES/COMMANDS

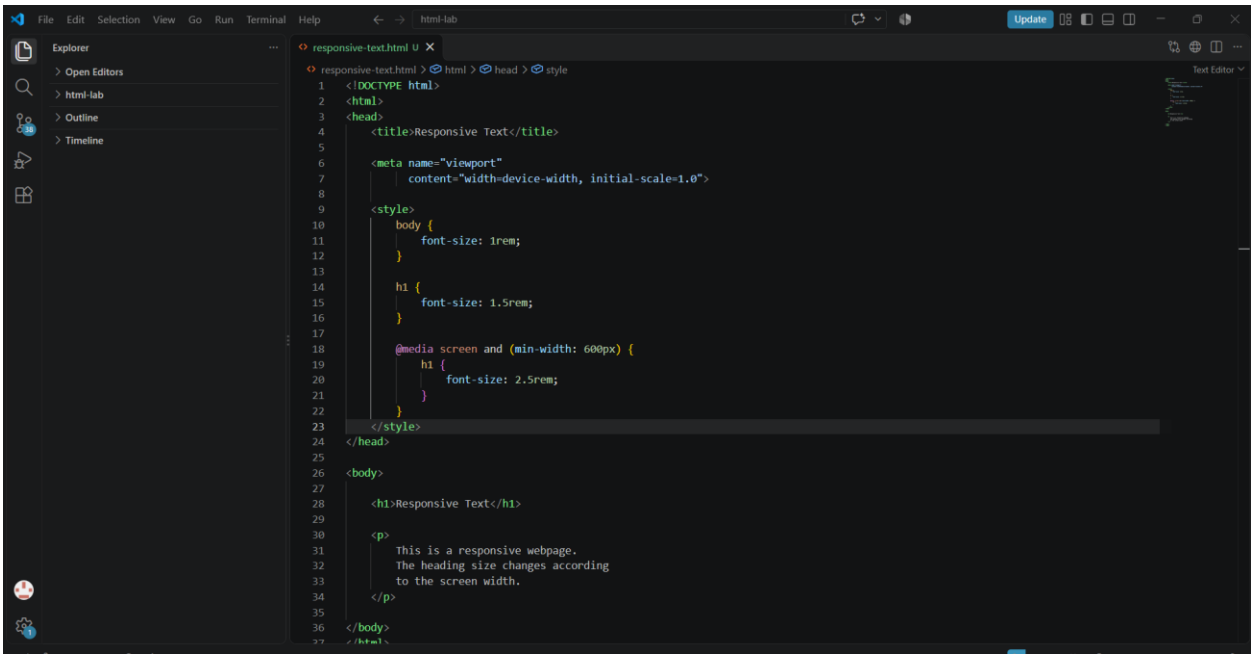
TASK 1 : CREATE mobile-first.html



TASK 2 : CREATE responsive-image.html



TASK 3 : CREATE responsive-text.html



6.INPUT

Task 1:

- Number of cards: 2
- Card headings: Card One and Card Two

- Mobile layout: Vertical
- Screen width: 600px and above (horizontal layout)

Task 2:

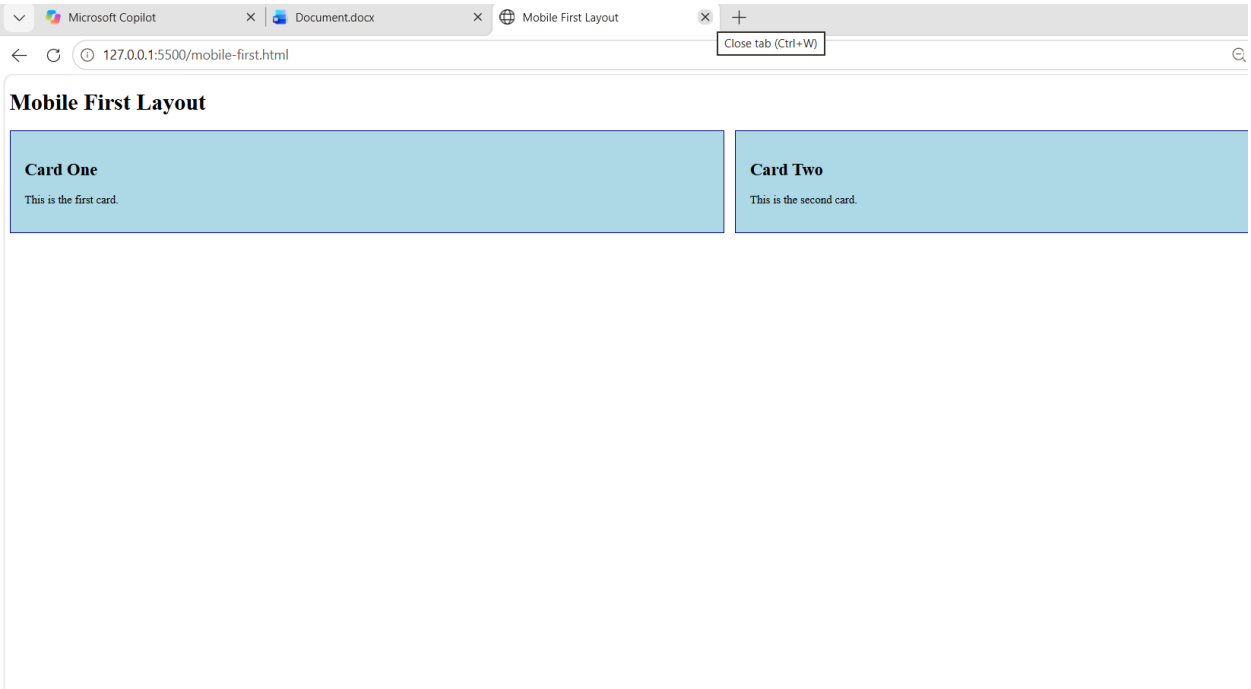
- Image file: Image provided by the instructor
- Maximum width: 100%
- Height: Auto

Task 3:

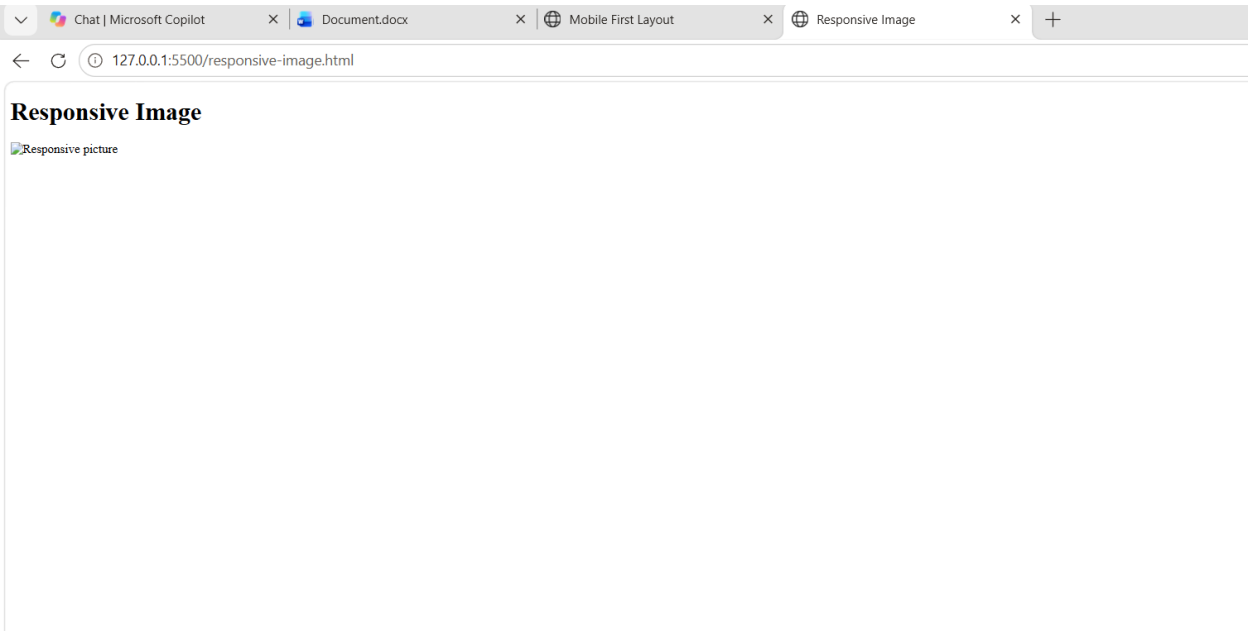
- Body font size: 1rem
- Initial heading size: 1.5rem
- Heading size at 600px and above: 2.5rem

7.OUTPUT

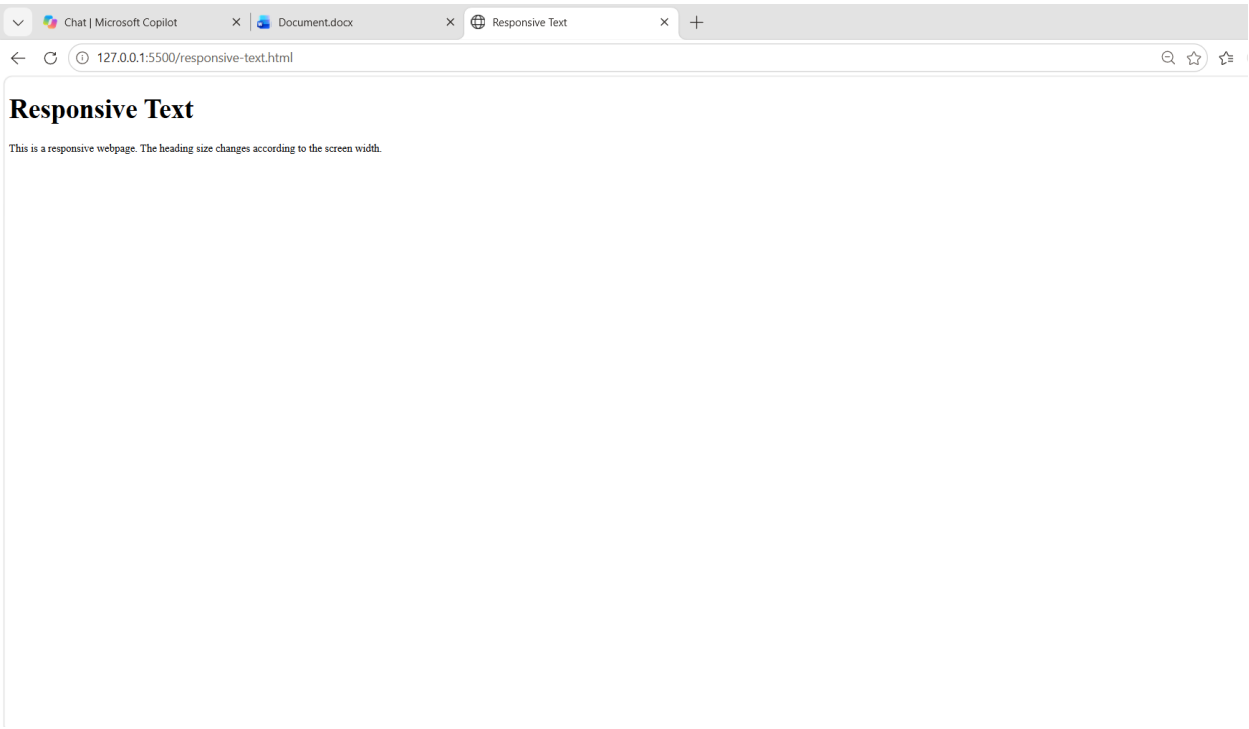
TASK 1 : Screenshot of `mobile-first.html` output



TASK 2 : Screenshot of `responsive-image.html` output.



TASK 3 : Screenshot of `responsive-text.html` output



8.RESULT

The experiment was performed successfully. Responsive webpages were created using CSS media queries. Flexible layouts, responsive images, and adjustable text sizes were implemented to improve webpage appearance on different screen sizes.